

TiMBL: Tilburg Memory-Based Learner

version 7.0

Reference Guide

Walter Daelemans* Jakub Zavrel† Ko van der Sloot††

Antal van den Bosch‡

(*) CLiPS - Computational Linguistics Group
Department of Linguistics
University of Antwerp

(†) Zeta Alpha

(††) Centre for Language Studies
Radboud University Nijmegen

(‡) Faculty of Humanities
Utrecht University

URL: <https://languagemachines.github.io/timbl>¹

July 3, 2026

¹This document is available from https://github.com/LanguageMachines/timbl/blob/master/docs/Timbl_7.0_Manual.pdf.

Contents

1	GNU General Public License	1
2	Installation	3
3	Changes	5
3.1	From version 6.4 to 7.0	5
3.2	From version 6.3 to 6.4	6
3.3	From version 6.2 to 6.3	6
3.4	From version 6.1 to 6.2	6
3.5	From version 6.0 to 6.1	7
3.6	From version 5.1 to 6.0	7
3.7	From version 5.0 to 5.1	8
3.8	From version 4.3 to 5.0	8
3.9	From version 3.0 to 4.3	9
3.10	From version 2.0 to 3.0	11
3.11	From version 1.0 to 2.0	12
4	Quick-start Tutorial	13
4.1	Data	13
4.2	Using TiMBL	14
4.3	Algorithms and metrics	17
4.4	More options	18
5	Memory-based learning algorithms	23
5.1	Memory-based learning	23

5.1.1	The Overlap metric	25
5.1.2	Information-gain and gain ratio feature weighting	26
5.1.3	Chi-squared and shared variance feature weighting	27
5.1.4	Modified value difference, Jensen-Shannon divergence, and Jeffrey divergence metrics	28
5.1.5	Dot-product and cosine metrics	30
5.1.6	Distance-weighted class voting	30
5.1.7	Tie breaking	32
5.1.8	Exemplar weighting	33
5.2	Indexing optimizations	33
5.2.1	Tree-based indexing	33
5.3	IGTree	34
5.4	The TRIBL and TRIBL2 hybrids	35
5.5	IB2: Incremental editing	36
5.6	Advanced evaluation metrics	37
5.7	Applications of TiMBL	39
6	Software usage and options	43
6.1	Command line options	43
6.1.1	Algorithm and metric selection	44
6.1.2	Input options	46
6.1.3	Output options	47
6.1.4	Internal representation options	48
6.1.5	Hidden options	49
6.2	File formats	50
6.2.1	Data files	50
6.2.2	Weight files	53
6.2.3	Value difference files	54
6.2.4	Tree files	55

Preface

Memory-Based Learning (MBL) is an elegantly simple and robust machine-learning method applicable to a wide range of tasks in Natural Language Processing (NLP). Starting from our research group at Tilburg University, we have been working since the end of the 1980s on the development of Memory-Based Learning techniques and algorithms. The foundations are bundled in Daelemans and Van den Bosch (2005). Section 5.7 provides a historical overview of work on the application of MBL in NLP. With the establishment of the ILK (Induction of Linguistic Knowledge) research group in 1997, and with the increasing use of MBL at the CNTS (now CLiPS) research group of the University of Antwerp, the need for a well-coded and uniform tool for our main algorithms became more urgent. TiMBL was the result of combining ideas from a number of different MBL implementations, cleaning up the interface, and using a whole bag of tricks to make it more efficient. We think it has become a useful tool for NLP research, and, for that matter, for many other domains where classification tasks are learned from examples, so we started to release the software in 1999. With the release of the sixth version of TiMBL we moved to releasing our software under the GPL license, for anyone to use, adapt and improve under the conditions stated in the license.

Memory-Based Learning is a direct descendant of the classical k -Nearest Neighbor (k -NN) approach to classification, which has become known as a powerful pattern classification algorithm for numeric data. In typical NLP learning tasks, however, the focus is on discrete data, very large numbers of examples, and many attributes of differing relevance. Moreover, classification speed is a critical issue in any realistic application of Memory-Based Learning. These constraints demand non-trivial data structures and speedup optimizations for the core k -NN classifier. Our approach has resulted in an architecture which compresses the typical flat file organization found in straightforward k -NN implementations, into a decision-tree structure. While the decision tree can be used to retrieve the exact k -nearest neighbors (as happens in the IB1 algorithm within TiMBL), it can also be deterministically traversed as in a decision-tree classifier (the method adopted by the IGTREE algorithm). We believe that our optimizations make TiMBL one of the fastest discrete k -NN implementations around.

The main effort in the development and maintenance of this software was and continues to be invested by Ko van der Sloot. The code started as a rewrite of `nibl`, a piece of software developed by Peter Berck from a Common Lisp implementation by Walter Daelemans of IB1-IG. Some of the index optimizations in TiMBL are due to Jakub Zavrel. The code has benefited substantially from trial, error and scrutiny by all past and present members of the ILK and CLiPS (formerly CNTS) groups in Tilburg and Antwerp. We are furthermore indebted to Ton Weijters of Eindhoven Technical University for his inspirational early work on k -NN and for his ideas on IGTREE.

Our sincere thanks go to the many users of TiMBL who have contributed to it immensely by giving us feedback and reporting bugs, and to the two organisations that have supported and enabled its development: NWO, the Netherlands Organization for Scientific Research, and the Tilburg School for Humanities and Digital Sciences. NWO funding has spanned three subsequent periods totalling fourteen years. From 1997 until 2001 development was part of the “Induction

of Linguistic Knowledge” research project funded by NWO’s *Pionier* programme. Between 2001 and 2006 it was funded as part of the “Memory Models of Language” research project under the NWO *Vernieuwingsimpuls* programme, and between 2006 and 2011 as part of the “Implicit Linguistics” research project under the NWO Vici programme.

The current release (version 7.0) is the result of a long series of incremental releases since version 6.4, which together brought, among other things, a multiprocessor option that “clones” TiMBL internally so that it can classify a test set on n available processors, improved Unicode/UTF-8 handling, and a number of new options. An elaborate description of the changes from version 1.0 up to 7.0 can be found in Chapter 3. Although all new features have been tested for some time in our research groups, the software may still contain bugs and inconsistencies in some places. We would appreciate it if you would send bug reports, ideas about enhancements of the software and the manual, and any other comments you might have, to our issue tracker at <https://github.com/LanguageMachines/timbl/issues>.

This reference guide is structured as follows. In Chapter 1 you can find the terms of the open source license according to which you are allowed to use TiMBL. The subsequent chapter gives some instructions on how to install the TiMBL package on your computer. Chapter 3 lists the changes that have taken place up to the current version. Next, Chapter 4 offers a quick-start tutorial for readers who want to get to work with TiMBL right away. The tutorial describes, step-by-step, a case study with a sample data set (included with the software) representing the linguistic domain of predicting the diminutive inflection of Dutch nouns. Readers who are interested in the theoretical and technical details of Memory-Based Learning and of this implementation can refer to Chapter 5. Chapter 6 provides full reference to the command line options of TiMBL and supported file formats.

Chapter 1

GNU General Public License

TiMBL is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 3 of the License, or (at your option) any later version.

TiMBL is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with TiMBL. If not, see <http://www.gnu.org/licenses/>.

In publication of research that makes use of TiMBL 7.0, a citation should be given of: *“Walter Daelemans, Jakub Zavrel, Ko van der Sloot, and Antal van den Bosch (2026). TiMBL: Tilburg Memory Based Learner, version 7.0, Reference Guide. Available from <https://languagemachines.github.io/timbl>”*

For information about commercial licenses for TiMBL 7.0, contact lamasoftware@science.ru.nl, or send your request in writing to:

Prof. dr. Antal van den Bosch
Utrecht University
Faculty of Humanities
P.O. Box 80125
3508 TC Utrecht
the Netherlands

Chapter 2

Installation

Before installing TiMBL from source, you should first check whether your operating system's package manager already offers an up-to-date package for it (TiMBL is available in, among others, Debian, Ubuntu and Homebrew). If so, that is by far the easiest route.

The home of the TiMBL project is on GitHub:

<https://github.com/LanguageMachines/timbl>

and its documentation and further information can be found at:

<https://languagemachines.github.io/timbl>

To build TiMBL from source, download the latest release tarball from the releases page (<https://github.com/LanguageMachines/timbl/releases>), or clone the repository directly with git. The source distribution contains the complete source code (C++), a few sample data sets, the license, and documentation. Building should be relatively straightforward on most UNIX-like systems (Linux and macOS).

To install from a release tarball, unzip the downloaded file (> is the command line prompt):

```
> tar xzf timbl-7.0.tar.gz
```

This will make a directory `timbl-7.0` under your current directory. (The numbering may be higher; the current guide refers to version 7.0).

Alternatively, to build the latest development version, clone the repository:

```
> git clone https://github.com/LanguageMachines/timbl.git
> cd timbl
> bash bootstrap.sh
```

TiMBL depends on a few other packages, most notably `ticcutils` (also available from the LanguageMachines GitHub organisation), `pkg-config`, `libc++` and `libxml2`. Install these through your distribution's package manager before configuring; alternatively, running `./build-deps.sh` will fetch and build the LanguageMachines dependencies for you.

Go to the source directory, and configure the package by typing

```
> ./configure --prefix=<location.to.install>
```

If you do not use the `--prefix` option, TiMBL will try to install itself in the directory `/usr/local/`. If you do not have `root` access you can specify a different installation location such as `$HOME/install`.

It is not obligatory to install TiMBL, but if you plan to install TiMBL-based extensions such as `TimblServer`¹, `Mbt`², or `Frog`³, or you want to build your own extensions using the TiMBL API, installing is the best choice.

After `configure` you can build TiMBL:

```
> make
```

and (as recommended) install:

```
> make install
```

If the process was completed successfully, you should now have an executable file named `timbl` (low caps) in the directory `<location.to.install>/bin`, and a static library `libTimbl.a` in the directory `<location.to.install>/lib`. Additionally, several demo programs named `api.test*`, `classify` and `tse` are created in the `./demos` subdirectory.

Within the `<location.to.install>` directory a subdirectory is also created: `share/doc/timbl` where the TiMBL 7.0 documentation can be found, and which in turn contains a subdirectory `examples` with example data files. Some of these data sets are used in the Quick Start Section 4 of this document; other data and source files are referred to in the API documentation. The latter, along with a pdf version of this document, can also be found in the `doc` directory. Note that the API documentation is a beta-state document.

TiMBL should now be ready for use. If you want to run the examples and demos from this manual, you should act as follows:

- Be sure to add `<location.to.install>/bin` to your `PATH`. In many shells something like

```
> export PATH=$PATH:<location.to.install>/bin
```

 will do.
- copy all the files from `<location.to.install>/share/doc/timbl/examples` to some working location. (By default, TiMBL writes its results to the directory where it finds the data.)
- and test:

```
cd to the working location, and then
timbl -f dimin.train -t dimin.test
```

If you did not install TiMBL, the executable can be found in the `src` directory of the build. The demo files can be found in the `demo` directory.

Problems with the installation, bug reports, comments and questions are welcome at our issue tracker, <https://github.com/LanguageMachines/timbl/issues>.

¹<https://github.com/LanguageMachines/timblserver>

²<https://languagemachines.github.io/mbt>

³<https://languagemachines.github.io/frog>

Chapter 3

Changes

This chapter gives a brief overview of the changes from all previously released versions (1.0 up to 7.0) for users already familiar with the program.

3.1 From version 6.4 to 7.0

Between version 6.4 and 7.0, TiMBL saw a long series of incremental releases. Most of these focused on internal code quality, build-system modernisation, and bug fixes; the changes most relevant to users are summarised below. The `NEWS` and `ChangeLog` files in the source repository give the complete per-release history.

- With `--prune`, an `IGTree` instance base can now be pruned directly from an `IB1/TRIBL2` instance base, instead of after a full training: TiMBL reads an existing instance base (`-i`), optionally adds new training material (`-f`), prunes the tree, and writes the result to a new file (`-I`).
- With `--limit <l>`, classification can be restricted to the `<l>` features with the highest weights; TiMBL automatically restarts with an adapted `-m` option that ignores the remaining features.
- The options `-f` (training file) and `-i` (instance base) may now be used without `-t`, that is, to build and store an instance base without immediately testing.
- A new `Tabbed` input format was added to the `-F` option, in which fields are separated by tab characters, so that spaces within field values are significant.
- TiMBL now handles Unicode (UTF-8) input robustly: strings are internally normalised to NFC, which makes the program reliable on text in a wide range of scripts, including non-Latin ones.
- The long options `--version` and `--help` are now accepted alongside `-V` and `-h`.

For developers using the TiMBL API, version 6.8 introduced a major refactoring that modernises the code (Unicode strings are now the default, and C-style arrays and casts were removed) and that breaks both the API and the ABI; code written against earlier versions may need to be adapted and recompiled. Building TiMBL now requires a C++17 compiler. The dependencies on

Boost and libtar have been removed; TiMBL relies on `ticcutils`, `libicu` and `libxml2`. Since version 6.4.7 the source repository is hosted on GitHub, and a `Dockerfile` and a `build-deps.sh` helper script are provided.

3.2 From version 6.3 to 6.4

- With `--clones=<n>`, TiMBL can be told to classify a test set using n processors in parallel. This option will speed up classification near-linearly with respect to n , although with larger n the overhead causes the speedup to be increasingly sub-linear.
- We added Jensen-Shannon divergence as a pairwise similarity metric for feature values. With MVD and Jeffrey divergence, TiMBL now offers three class distribution distance functions for value differences.
- Euclidean distance was long overdue as an optional distance function (`-mE`) for numeric features.
- `-w 5` or `-w SD` sets the feature weighting of numeric features to the standard deviation of its values.
- With `--occurrences={train|test|both}`, instances in instance base files can be accompanied by a (long) integer specifying a number of token occurrences of that instance, so that it does not count as a single instance in the TiMBL tree, but as $<n>$ instances. These counts can be exclusively present in the training set, in the test set, or in both if a training and test action is invoked at once.
- The verbosity option `+v b` adds overall statistics of IGTREE nodes (leafs and non-ending nodes) and branching at each tree level.
- The executable file is now called `timbl` instead of `Timbl` to be more GNU/Linux compliant. Previous users of TiMBL may want to check scripts for this name change.

3.3 From version 6.2 to 6.3

- All server-related functionality is removed from TiMBL. A new `TimblServer` package is available which provides the same interface as TiMBL up to version 6.3, but also adds some extra features, such as running multiple but separate experiments on one TCP port. See the `TimblServer` package for more details.
- Starting with TiMBL 6.3 we support installable packages for Debian and Ubuntu (`.deb`), RedHat (`.rpm`) and macOS¹.
- Some bugs and inconsistencies have been fixed.

3.4 From version 6.1 to 6.2

Version 6.2 differs from 6.1 in a great number of internal changes aimed at making the code better maintainable and extendible, in some minor bug fixes, and in the following more prominent changes:

¹Up-to-date packaging information is available from <https://languagemachines.github.io/timbl>

- A new distance metric, the Dice coefficient, has been added; the metric can be set with `-mDC`. Analogous to the Levenshtein (`-mL`) metric, the Dice coefficient operates at the feature value level; it computes the overlap in character bigrams of two value strings.
- Value difference matrices, as used by the MVDM and Jeffrey divergence distance metrics, can now be written to file, and read into TiMBL, allowing for user-defined value difference metrics to be used. The new command line options are `--matrixout=<filename>` and `--matrixin=<filename>`.
- The IGTREE algorithm has been optimized beyond the improvements introduced in version 6.0. With very large training sets, IGTREE was reported to be exponentially slower in the later stages of training. Trees are now built in near-linear time.

Finally, besides minor bug fixes, a great number of internal changes were made to make the code better maintainable and extendible.

3.5 From version 6.0 to 6.1

Version 6.1 differs from 6.0 mainly in the changed configuration. It is now based on autotools and is delivered as an installable package. Some bugs have been fixed as well.

3.6 From version 5.1 to 6.0

Version 6.0 differs from 5.1 firstly in terms of internal changes aimed at increasing classification speed and lowering memory usage, of which the most prominent are

- The IGTREE algorithm has been optimized. Learning has been made more memory-lean, while classification has been optimized so that it is now orders of magnitude faster than before on most data sets.
- MVDM matrices are partly prestored; only the MVDM values of pairs of frequent values are precomputed. The threshold frequency n can now be determined with `-c n`. This way memory can be traded for speed, up to a point. The default value 10 remains the recommended one.

Also, two metrics and several verbosity options and other command-line switches are added:

- Two distance metrics are added: `-mC` sets the Cosine metric, and `-mL` sets the Levenshtein metric. The latter metric operates at the feature-value level, and thus offers an alternative to the all-or-nothing Overlap metric for string-valued features.
- Class distribution output generated with `+v db` can be normalized so that they add to 1.0, with the additional `-G` option (or `-G0`). As a simple smoothing option, with `-G1:double` all class votes are incremented by `double` before normalization. For example, `-G1:1` (or `-G1` for short) is “add one”-smoothing; `-G1:0.5` adds 0.5 to all class votes.
- With `-Beam=n` (from version 6.2 onwards: `--Beam=n`), where n is an integer, the `+v db` output is constrained to the n classes receiving the highest votes. This special limit is useful in cases in which the `+v db` output, typically used for further processing, generates far too much output in its default unconstrained setting.

- Class distributions are not stored on non-terminal nodes with IGTREE and TRIBL by default. To revert this default, e.g. to be able to use `+v db` with IGTREE, the setting `+D` can be used.
- With `-T n`, the user can specify that the n th column in the training set of labeled examples contains the label to be predicted, while all other columns represent the input features. By default, the final column is assumed to contain the class labels.
- After classification, TiMBL reports its classification speed at microsecond precision instead of in seconds.
- The verbosity option `+v md` displays the level at which a classification was made by IGTREE (`-al`), and whether the class label was obtained from a leaf node or an end node.
- With `-X [file]`, TiMBL dumps its internal TiMBL tree into a file containing an XML tree. This option is analogous to `-I [file]`, which prints a TiMBL tree in TiMBL's proprietary format, the difference being that the latter format can be read into TiMBL again.
- Several minor bugs have been resolved.

3.7 From version 5.0 to 5.1

Version 5.1 adds speed and memory improvements that are notable with datasets that have very large amounts of examples, features, feature values, or classes (and, especially, combinations of those). Previous versions exhibited exponential slowdown in some worst cases; this has been largely countered. On the outside, TiMBL has been updated in the following aspects:

- TiMBL offers extended performance reporting: next to accuracy it reports on micro and macro-averages of F-score and AUC (area under the ROC-curve) with `+v as`. Optionally, it also shows each individual class' precision, recall (or true positive rate), and false positive rate with `+v cs`.
- TiMBL always uses gain ratio feature weighting as the default case, if not specified by the user, also with the MVDM and Jeffrey Divergence similarity metrics.
- Two additional feature orderings for the internal TiMBL trees are added, `-TGxE` and `-TIxE` (gain ratio $\times$ entropy and information gain $\times$ entropy, respectively) to potentially tackle the problem of unbalanced trees.
- Bugs in leave-one-out testing with numeric features and with exemplar weighting were fixed.

3.8 From version 4.3 to 5.0

Version 5.0 is the conclusion of a number of recodings (mostly involving more generic treatment of variables to improve robustness, but also the removal of inverted indexing on the internal tree representation) that have changed the internals of TiMBL considerably. On the outside, TiMBL displays the following new characteristics:

- Next to the Overlap, MVDM, and Numeric distance functions, TiMBL now features the Jeffrey divergence distance function and the Dot-product distance function.

- The exponential-decay distance weighting function can be set using a second parameter, which can change the shape of the function from normal exponential to bell-shaped.
- In addition to the “binary” format, TiMBL can now read a more generic sparse data format. This format allows instances to be coded by tuples of $\langle \text{feature number, feature value} \rangle$ where the value can be symbolic or numeric rather than only binary.
- Tree files generated by TiMBL versions 1.*, 2.* and 3.* are no longer supported.
- The command line interface has had the following additions, including the ones reflecting the above changes:
 - `-mJ` activates the Jeffrey divergence distance metric.
 - `-mD` activates the Dot-product distance metric.
 - `-dED : <a> : <b>` (without whitespace) sets the α and new β parameters. If unspecified, as in `-d ED : <a>` or the older (deprecated) `-d ED <a>`, β is set to 1.0.
 - `-F Sparse` declares that training and test files are in the sparse $\langle \text{feature number, feature value} \rangle$ tuple-format described in more detail in section 6.1.
 - `+v k` is a new verbosity option that prints all class distributions per k -nearest distance per classified instance in the output file. It works analogous to the `+v n` option, but does not print the neighbors themselves.

3.9 From version 3.0 to 4.3

As the last upgrade of the version 4 strain, version 4.3 added some command line functionality and internal code changes to version 4.2. Minor progressive changes from 4.0 to 4.3 are found at the bottom of this list and are marked as such.

- Distance weighting of the k nearest neighbors. This classical exemplar weighting scheme (Dudani, 1976) allows closer nearest neighbors in the k to have a more prominent vote in classification. TiMBL incorporates linear, inversed, and exponential distance weighting.
- Incremental edited memory-based learning with IB2 (Aha, Kibler, and Albert, 1991). This incremental version of IB1 adds instances to memory only when those instances are misclassified by the then-current set of instances in memory.
- Frequency-filtered MVDM distance metric. The option, which is not selected by default, is an add-on of the MVDM metric, that backs off from the MVDM metric to the Overlap distance function whenever one or both in a pair of matched values occurs fewer times in the training material than a user-determined threshold.
- TRIBL2. The TRIBL2 algorithm has been implemented as an additional trade-off between IGTREE and IB1. In contrast to TRIBL, TRIBL2 uses no threshold parameter.
- Exemplar weighting. TiMBL can read additional numeric exemplar weights (generated externally) when reading a data file, and use these weights during neighbor distance computation in k -NN classification.
- Cross-validation testing. Analogous to the leave-one-out testing option, with cross-validation testing it is possible to let TiMBL run systematic tests on different values of parameters, without completely re-initializing the classifier in every fold of the validation experiment.
- The number of concurrent connections to a TiMBL server has been restricted, but can be set to different values.

- The command line interface has had several additions reflecting the above changes, plus one extra verbosity option:
 - the `-d metriccode` option sets the distance weighting metric. Three metrics are available: inverse distance (code ID), inverse linear (IL), and exponential decay (ED, which takes an extra argument *a*, without whitespace, determining the factor of the exponential function). By default, no distance weighting is used (code Z). See Chapter 5 for descriptions.
 - the `-L n` option sets the frequency threshold in the optional switch (backoff) from MVDM or Jeffrey divergence to Overlap; whenever in an MVDM or Jeffrey divergence distance computation one or both of a pair of values occur fewer than *n* times, Overlap is used rather than the MVDM metric. The default value for *n* is 1 (no switching).
 - the `-a 3` or `-a IB2` switch invokes the IB2 algorithm. This algorithm expects to have the `-b` switch set.
 - the `-b n` option sets the number (*n*) of lines counting from the top of the training set file, which form the bootstrap set of memorized instances to which IB2 will start adding instances incrementally.
 - the `-a 4` or `-a TRIBL2` switch invokes the TRIBL2 algorithm.
 - the `-C n` switch (default: *n* set to 10) restricts the number of concurrent connections to a TiMBL server (cf. the `-S` switch).
 - the `+v/-v` option has `cm` as a new optional argument; it returns the confusion matrix, obtained after testing, between predicted and actual classes in the test data.
- The “programmer’s reference” or API section has been separated from this manual. This new API, describing the underlying structure of TiMBL, is available as a separate document in the TiMBL software distribution.
- Two bugs relating to a type of sparse data problem have been resolved. The first involved leave-one-out experiments on data sets with features that have values that occur only once in the training data. The second bug occurred with the use of the `-F Binary` option with the same type of data.
- [4.1] Exemplar weights are stored in the TiMBL-tree.
- [4.1] The core representation of TiMBL-trees has been modified, causing no changes at the surface except that the TRIBL variant uses less memory.
- [4.2] Feature value and class information in the internal TiMBL tree is hashed, by default, except with binary features. Hashing can be explicitly set on or off through the flag `+H` or `-H`.
- [4.2] The discretization of numeric features, used for computing feature weights, has changed from linear binning between minimum and maximum values, to equal-content binning.
- [4.2] Tie resolution between equal class distributions in the nearest neighbors set is resolved by first expanding the *k* by one value. If the tie persists after the enlargement of the nearest neighbor set, the original tie resolution method is applied.
- [4.3] Internal changes in the code (with no effect on learning and classification functionality) have been implemented with respect to namespaces.
- [4.3] A progress marker (one dot per 10 seconds) in computationally intensive operations on the internal representation of the instance base (e.g. pruning IGTREES) is added in TiMBL’s screen output.
- A number of bugs have been fixed, notably to handle erroneous input more robustly.

3.10 From version 2.0 to 3.0

- **Server functionality.** Apart from the standard processing of test items from a file, alternatively you can now specify a portnumber with `-S portnumber` to open a socket and send commands for classification of test patterns or change of parameters to it. A sample client program is included in the distribution. This allows fast response times when small amounts of test material are presented at various intervals. It also opens the possibility of having large numbers of “classification agents” cooperate in real time, or of classification of the same data with different parameters.
- **Leave-one-out testing.** To get an estimate of the classification error, without setting aside part of one’s data as a test set, one can now test by “leave-one-out” (`-t leave_one_out`), in effect testing on every case once, while training on the rest of the cases, without completely re-initializing the classifier for every test case.
- **Support for sparse binary features.** For tasks with large numbers of sparse binary features, TiMBL now allows for an input format which lists only the “active” features, avoiding the listing of the many (zero-valued) features for each case. This format is described in Section 6.2.1.
- **Additional feature weighting metrics.** We have added chi-squared and shared variance measures as weighting schemes. These weighting metrics are sometimes more robust to large numbers of feature values and other forms of data sparseness.
- **Different metrics (Overlap, MVDm or Numeric) can be applied to different features.**
- **The command line interface has slightly been cleaned up, and re-organized:**
 - The `-m metricnumber` switch to choose metrics has been replaced by the use of a specification string following `-m`. E.g. you can specify to use MVDm as the default metric, but use Overlap on features 5-7,9, Numeric on feature 1, and ignore feature 10 (`-m M:05-7,9:N1:I10`).
 - All of the output needed for analysing the matching of nearest neighbors has been moved to the verbosity setting.
 - Verbosity levels and some other options can be switched on `+v` and off `-v`, even between different classification actions.
 - Because of the large amount of verbosity levels, the `+v` option takes mnemonic abbreviations as arguments instead of numeric verbosity levels. Although the old (numeric) format is still supported, it’s use is not encouraged as it will disappear in future versions.
- **Because of significant optimizations in the nearest neighbor search, the default is no longer to use inverted indexes.** These can however still be turned on by using the `+-` switch on the command line.
- **You can now choose the output filename or have it generated by TiMBL on the basis of the test filename and the parameters.**
- **You can use TiMBL in a pipeline of commands by specifying ‘-’ as either input, output or both.**
- **Several problems with the display of nearest neighbors in the output have been fixed.**
- **The API has been adapted a bit to allow more practical use of it.**

3.11 From version 1.0 to 2.0

- We have added a new algorithm: `TRIBL`, a hybrid between the fast `IGTREE` algorithm and real nearest neighbor search (for more details, see 5.4, or Daelemans, Weijters, and Van den Bosch (1997b)). This algorithm is invoked with the `-a 2` switch and requires the specification of a so-called `TRIBL-offset`, the feature where `IGTREE` stops and case bases are stored under the leaves of the constructed tree.
- Support for numeric features. Although the package has retained its focus on discrete features, it can now also process numeric features, scale them, and compute feature weights on them. You specify which features are numeric with the `-N` option on the command line.
- The organization of the code is much more object-oriented than in version 1.0.
- A Memory-Based Learning API is made available. You can define Memory-Based classification objects in your own C++ programs and access all of the functionality of `TiMBL` by linking to the `TiMBL` library.
- It has become easier to examine the way decisions are made from nearest neighbors, because several verbosity-levels allow you to dump similarity values (`-D`), distributions (`-v 16`), and nearest neighbor sets (`-v 32`) to the output file. The `-d` option for writing the distributions no longer exists.
- Better support for the manipulation of `MVDM` matrices. Using the `-U` and `-u` options it is now possible to respectively save and read back value difference matrices (see Section 6.2.3).
- Both “pre-stored” and “regular” `MVDM` experiments now generate filenames with “`mvd`” in the suffix. This used to be “`pvd`” and “`mvd`” respectively.
- a number of minor bugs have been fixed.

Chapter 4

Quick-start Tutorial

This quick-start tutorial is meant to get you started with TiMBL right away. We discuss how to format the data of a task to serve as training examples, which choices can be made during the construction of the classifier, how various choices can be evaluated in terms of their generalization accuracy, and various other practical issues. The reader who is interested in more background information on TiMBL implementation issues and a formal description of Memory-Based Learning, is advised to read Chapter 5.

Memory-Based Learning (MBL) is based on the idea that intelligent behavior can be obtained by analogical reasoning, rather than by the application of abstract *mental rules* as in rule induction and rule-based processing. In particular, MBL is founded in the hypothesis that the extrapolation of behavior from stored representations of earlier experience to new situations, based on the similarity of the old and the new situation, is of key importance.

MBL algorithms take a set of examples (fixed-length patterns of feature-values and their associated class) as input, and produce a *classifier* which can classify new, previously unseen, input patterns. Although TiMBL was designed with linguistic classification tasks in mind, it can in principle be applied to any kind of classification task with symbolic or numeric features and discrete (non-continuous) classes for which training data is available. As an example task for this tutorial we go through the application of TiMBL to the prediction of Dutch diminutive suffixes. The necessary data sets are included in the TiMBL distribution, so you can replicate the examples given below on your own system.

4.1 Data

The operation of TiMBL will be illustrated below by means of a real natural language processing task: prediction of the diminutive suffix form in Dutch (Daelemans, Berck, and Gillis, 1997). In Dutch, a noun can receive a diminutive suffix to indicate *small size* literally or metaphorically attributed to the referent of the noun; e.g. *mannetje* means *little man*. Diminutives are formed by a productive morphological rule which attaches a form of the Germanic suffix *-tje* to the singular base form of a noun. The suffix shows variation in its form (Table 4.1). The task we consider here is to predict which suffix form is chosen for previously unseen nouns on the basis of their form.

For these experiments, we collect a representation of nouns in terms of their syllable structure as training material¹. For each of the last three syllables of the noun, four different features are

¹These words were collected from the CELEX lexical database (Baayen, Piepenbrock, and van Rijn, 1993).

Noun	Form	Suffix
huis (house)	huisje	-je
man (man)	mannetje	-etje
raam (window)	raampje	-pje
woning (house)	woninkje	-kje
baan (job)	baantje	-tje

Table 4.1: Allomorphic variation in Dutch diminutives.

collected: whether the syllable is stressed or not (values - or +), the string of consonants before the vocalic part of the syllable (i.e. its onset), its vocalic part (nucleus), and its post-vocalic part (coda). Whenever a feature value is not present (e.g. a syllable does not have an onset, or the noun has less than three syllables), the value '=' is used. The class to be predicted is either E (-etje), T (-tje), J (-je), K (-kje), or P (-pje).

Some examples are given below (the word in the rightmost column is only provided for convenience and is not used). The values of the syllabic content features are given in phonetic notation.

+	b	i	=	-	z	@	=	-	m	A	nt	J	<i>biezenmand</i>
=	=	=	=	=	=	=	=	+	b	I	x	E	<i>big</i>
=	=	=	=	+	b	K	=	-	b	a	n	T	<i>bijbaan</i>
=	=	=	=	+	b	K	=	-	b	@	l	T	<i>bijbel</i>

Our goal is to use TiMBL in order to train a classifier that can predict the class of new, previously unseen words as correctly as possible, given a set of training examples that are described by the features given above. Because the basis of classification in TiMBL is the storage of all training examples in memory, a test of the classifier's accuracy must be done on a separate test set. We will call these datasets `dimin.train` and `dimin.test`, respectively. The training set `dimin.train` contains 2999 words and the test set contains 950 words, none of which are present in the training set. Although a single train/test partition suffices here for the purposes of explanation, it does not factor out the bias of choosing this particular split. Unless the test set is sufficiently large, a more reliable generalization accuracy measurement is used in real experiments, e.g. 10-fold cross-validation (Weiss and Kulikowski, 1991). This means that 10 separate experiments are performed, and in each "fold" 90% of the data is used for training and 10% for testing, in such a way that each instance is used as a test item exactly once. Another reliable way of testing the real error of a classifier is leave-one-out (Weiss and Kulikowski, 1991). In this approach, every data item in turn is selected once as a test item, and the classifier is trained on all remaining items. Accuracy of the classifier is then the number of data items correctly predicted. With the option `-t leave_one_out`, this testing methodology is used by TiMBL. We will use this option in the tutorial on the file `dimin.data`, the union of `dimin.train` and `dimin.test`.

4.2 Using TiMBL

Different formats are allowed for training and test data files. TiMBL is able to guess the type of format in most cases. We will use comma-separated values here, with the class as the final value. This format is called C4.5 format in TiMBL because it is the same as that used in Quinlan's well-known C4.5 program for learning decision trees (Quinlan, 1993). See Section 6.2 for more information about this and other file formats.

An experiment is started by executing TiMBL with the two files (`dimin.train` and `dimin.test`)

as arguments ("`>`" is the command line prompt):

```
> timbl -f dimin.train -t dimin.test
```

Upon completion, a new file has been created with name `dimin.test.IB1.O.gr.kl.out`, which is identical to the input test file except that an extra comma-separated column is added with the class predicted by TiMBL. The name of the file provides information about the MBL algorithms and metrics used in the experiment (the default values in this case). We will describe these shortly.

Apart from the result file, information about the operation of the algorithm is also sent to the standard output. It is therefore advisable to redirect the output to a file in order to make a log of the results.

```
> timbl -f dimin.train -t dimin.test > dimin-expl
```

The defaults used in this case work reasonably well for most problems. We will now provide a point by point explanation of what goes on in the output.

```
TiMBL 7.0 (c) CLS/ILK/CLiPS 1998 - 2026.
Tilburg Memory Based Learner
Centre for Language Studies, Radboud University Nijmegen
CLiPS Computational Linguistics Group, University of Antwerp
Tue Jun 23 12:00:00 2026
```

```
Examine datafile 'dimin.train' gave the following results:
Number of Features: 12
InputFormat       : C4.5
```

TiMBL has detected 12 features and the C4.5 input format (comma-separated features, class at the end).

```
Phase 1: Reading Datafile: dimin.train
Start:      0 @ Mon Oct 19 21:30:00 2009
Finished:   2999 @ Mon Oct 19 21:30:00 2009
Calculating Entropy      Mon Oct 19 21:30:00 2009
Lines of data           : 2999
DB Entropy              : 1.6178929
Number of Classes       : 5
```

```
Feats Vals InfoGain GainRatio
1      3 0.030971064 0.024891536
2     50 0.060860038 0.027552191
3     19 0.039562857 0.018676787
4     37 0.052541227 0.052620750
5      3 0.074523225 0.047699231
6     61 0.10604433 0.024471911
7     20 0.12348668 0.034953203
8     69 0.097198760 0.043983864
9      2 0.045752381 0.046816705
10    64 0.21388759 0.042844587
11    18 0.66970458 0.18507018
12    43 1.2780762 0.32537181
```

```
Feature Permutation based on GainRatio/Values :
< 9, 5, 11, 1, 12, 7, 4, 3, 10, 8, 2, 6 >
```

Phase 1 is the training data analysis phase. Time stamps for start and end of analysis are provided. Some preliminary analysis of the training data is done: number of training items, number

of classes, entropy of the training data. For each feature, the number of values, and four variants of an information-theoretic measure of feature relevance are given. These are used both for memory organization during training and for feature relevance weighting during testing (see Chapter 5). Finally, an ordering (permutation) of the features is given. This ordering is used for building the tree-index to the case-base.

```
Phase 2: Learning from Datafile: dimin.train
Start:      0 @ Mon Oct 19 21:30:00 2009
Finished:   2999 @ Mon Oct 19 21:30:00 2009

Size of InstanceBase = 19231 Nodes, (769240 bytes), 49.77 % compression
Examine datafile 'dimin.test' gave the following results:
Number of Features: 12
InputFormat       : C4.5
```

Phase 2 is the learning phase: all training items are stored in an efficient way in memory for use during testing. Again timing information (real time) is provided, as well as information about the size of the data structure representing the stored examples and the amount of compression achieved.

```
Starting to test, Testfile: dimin.test
Writing output in:      dimin.test.IB1.O.gr.k1.out
Algorithm      : IB1
Global metric  : Overlap
Deviant Feature Metrics: (none)
Weighting      : GainRatio
Feature 1      : 0.024891535617620
Feature 2      : 0.027552191321752
Feature 3      : 0.018676787182524
Feature 4      : 0.052620750282779
Feature 5      : 0.047699230752236
Feature 6      : 0.024471910753751
Feature 7      : 0.034953203413051
Feature 8      : 0.043983864437713
Feature 9      : 0.046816704745507
Feature 10     : 0.042844587034556
Feature 11     : 0.185070180760327
Feature 12     : 0.325371814230901

Tested:      1 @ Fri Dec 24 20:27:07 2010
Tested:      2 @ Fri Dec 24 20:27:07 2010
Tested:      3 @ Fri Dec 24 20:27:07 2010
Tested:      4 @ Fri Dec 24 20:27:07 2010
Tested:      5 @ Fri Dec 24 20:27:07 2010
Tested:      6 @ Fri Dec 24 20:27:07 2010
Tested:      7 @ Fri Dec 24 20:27:07 2010
Tested:      8 @ Fri Dec 24 20:27:07 2010
Tested:      9 @ Fri Dec 24 20:27:07 2010
Tested:     10 @ Fri Dec 24 20:27:07 2010
Tested:     100 @ Fri Dec 24 20:27:07 2010
Ready:      950 @ Fri Dec 24 20:27:07 2010
Seconds taken: 0.0678 (14003.54 p/s)

overall accuracy:      0.967368 (919/950), of which 39 exact matches
There were 5 ties of which 4 (80.00%) were correctly resolved
```

In Phase 3, the trained classifier is applied to the test set. Because we have not specified which algorithm to use, the default settings are used (IB1 with information-theoretic feature weighting). This algorithm computes the similarity between a test item and each training item in terms of *weighted overlap*: the total difference between two patterns is the sum of the relevance weights of those features which are not equal. The class for the test item is decided on the basis of the least

distant item(s) in memory. To compute relevance, Gain Ratio is used (an information-theoretic measure, see Section 5.1.2). Time stamps indicate the progress of the testing phase. Finally, accuracy on the test set is logged, and the number of exact matches² and ties (two or more classes are equally frequent in the nearest neighbor set). In this experiment, the diminutive suffix form of 96.7% of the new words was correctly predicted. Train and test set overlap in 39 items, and the algorithm had to break five ties, four of which were broken correctly.

The meaning of the output file names can be explained now:

`dimin.test.IB1.O.gr.k1.out` means output file (`.out`) for `dimin.test` with algorithm MBL (=IB1), similarity computed as *weighted overlap* (`.O`), relevance weights computed with *gain ratio* (`.gr`), and number of most similar memory patterns on which the output class was based equal to 1 (`.k1`).

4.3 Algorithms and metrics

A precise discussion of the different algorithms and metrics implemented in TiMBL is given in Chapter 5. We will discuss the effect of the most important ones on our data set.

A first choice in algorithms is between using IB1 and IGTREE. In the trade-off between generalization accuracy and efficiency, IB1 usually, but not always, leads to more accuracy at the cost of more memory and slower computation, whereas IGTREE is a fast heuristic approximation of IB1, but sometimes less accurate. The IGTREE algorithm is used when `-a 1` is given on the command line, whereas the IB1 algorithm used above (the default) would have been specified explicitly by `-a 0`.

```
> timbl -a1 -f dimin.train -t dimin.test
```

We see that IGTREE performs only slightly worse (96.6%) than IB1 (96.8%) for this train-test partitioning of the data — it uses less memory and is faster, however.

When using the IB1 algorithm, there is a choice of metrics for influencing the definition of similarity. With *weighted overlap*, each feature is assigned a weight, determining its relevance in solving the task. With the *modified value difference metric* (MVDM), each pair of values of a particular feature is assigned a value difference. The intuition here is that in our diminutive problem, for example, the codas *n* and *m* should be regarded as being more similar than *n* and *p*. These pairwise differences are computed for each pair of values in each feature (see Section 5.1.4). Selection between weighted overlap and MVDM is done by means of the `-mM` parameter. The following selects MVDM, whereas `-mO` (*weighted overlap*) is the default.

```
> timbl -mM -f dimin.train -t dimin.test
```

Especially when using MVDM, but also in other cases, it may be useful to extrapolate not just from the most similar example in memory, which is the default, but from several. This can be achieved by using the `-k` parameter followed by the wanted number of nearest neighbors. E.g., the following applies IB1 with the MVDM metric, with extrapolation from the 5 nearest neighbors.

```
> timbl -mM -k5 -f dimin.train -t dimin.test
```

²An exact match in this experiment can occur when two different nouns have the same feature-value representation.

	no weight (overlap)	gain ratio	information gain	chi squared
Overlap, $-k1$	86.4	96.7	96.6	96.6
Overlap, $-k3$	73.1	96.4	96.8	96.9
Overlap, $-k5$	62.6	95.4	96.1	96.1
MVDM, $-k1$	95.8	96.3	96.1	96.2
MVDM, $-k3$	97.4	97.6	97.6	97.6
MVDM, $-k5$	97.8	97.7	97.7	97.7

Table 4.2: Some results for diminutive prediction.

Whenever more than one nearest neighbor is taken into account for extrapolation, it may be useful to weigh the influence of the neighbors on the final decision as a function of their distance from the test item. Several possible implementations of this distance function are provided. E.g., the following provides inverse distance:

```
> timbl -mM -k5 -dID -f dimin.train -t dimin.test
```

Within the IB1 *weighted overlap* option, the default feature weighting method is gain ratio. Other feature relevance weighting methods are available as well. By setting the parameter $-w$ to 0, an *unweighted overlap* definition of similarity is created where each feature is considered equally relevant. In that case, similarity reduces to the number of equal values in the same position in the two patterns being compared. As an alternative weighting, users can provide their own weights by using the $-w$ parameter with a filename in which the feature weights are stored (see Section 6.2.2 for a description of the format of the weights file).

Table 4.2 shows a small matrix indicating the effect of distance metric (Overlap versus MVDM) and weighting method choice on generalization accuracy, using the same training and test set as before, and increasing k from 1 to 3 and 5. While increasing k leads to a deterioration of generalization accuracy with the Overlap function, it leads to improvements with MVDM. Another clear contrast is that the absence of feature weighting leads to the lowest scores with the Overlap function, and the highest score with MVDM and $k = 5$. Given that TiMBL offers several more hyperparameters than only k , the distance metric, and the feature weighting metric, it should be obvious that even with a single training and test set experiment, a large experimental matrix can be explored. Unfortunately, the location of the cell with the highest number in this matrix cannot be predicted upfront. It is therefore useful to try out a large set of reasonable combinations of options by cross-validation on the training data to achieve best results with MBL (Van den Bosch, 2004b). The option $-t @f$ where f is the name of a file, allows you to predefine various combinations of options to be tested and test them without having the training stages repeated each time. See Chapter 6.1.

4.4 More options

Several input and output options exist to make life easier while experimenting. See Chapter 6.1 for a detailed description of these options. One especially useful option for testing linguistic hypotheses is the ignore option, which allows you to skip certain features when computing similarity. E.g. if we want to test the hypothesis that only the rime (nucleus and coda) and the stress of the last syllable are actually relevant in determining the form of the diminutive suffix,

we can execute the following with the previously best parameter settings to disregard all but the fourth-last and the last two features. As a result we get an accuracy of 97.1%.

```
> timbl -mM:I1-8,10 -k5 -w0 -f dimin.train -t dimin.test
```

The `+/-v` (verbosity) option allows you to control the amount of information that is generated in the output, ranging from nearly nothing (`+v s`) to a lot (`+v as+cs+di+db+n+k`). Specific verbosity settings exist for dumping option settings (`+v o`), feature relevance weights (default), value-class conditional probabilities (`+v p`), exact matches (`+v e`), distributions (`+v db`), a confusion matrix (`+v cm`), advanced statistics besides accuracy: micro-average and macro-average F-score and AUC (`+v as`), per-class advanced statistics (`+v cs`), the nearest neighbors on which decision are based (`+v n`), just the class distributions per k -nearest distance per classified instance (`+v k`), or the distances to the nearest neighbor (`+v di`). E.g. the following command results in an output file with distributions.

```
> timbl +v db -f dimin.train -t dimin.test
```

The resulting output file `dimin.test.IB1.O.gr.k1.out` contains lines like the following.

```
+ ,t,L,=-,m,@,=-,l,I,N,E,E { E 1.00000 }
=,=,=,=,=,=,=,+,pr,O,p,J,J { E 3.00000, J 12.0000 }
=,=,=,=,=,=,=,+,w,e,t,J,J { J 2.00000 }
=,=,=,=,+,t,L,n,-,h,L,s,J,J { J 1.00000 }
=,=,=,=,=,=,=,+,t,L,n,T,T { T 1.00000 }
=,=,=,=,=,=,=,+,z,o,m,P,P { P 3.00000 }
+,d,a,=-,m,@,s,-,kr,A,ns,J,J { J 1.00000 }
=,=,=,=,+,a,rd,-,m,A,n,E,E { E 2.00000 }
=,=,=,=,=,=,=,+,f,M,n,T,T { T 43.0000, E 20.0000 }
-,d,u,=-,k,@,=-,m,A,nt,J,J { J 1.00000 }
```

This information can e.g. be used to assign a certainty to a decision of the classifier, or to make available a second-best back-off option. Another verbosity option, `+v di`, displays the distance to the nearest neighbor:

```
> timbl +v di -f dimin.train -t dimin.test

+,l,a,=-,d,@,=-,k,A,st,J,J      0.070701
-,s,i,=-,f,E,r,-,st,O,k,J,J      0.000000
=,=,=,=,=,=,=,+,sp,a,n,T,T      0.042845
=,=,=,=,=,=,=,+,st,o,t,J,J      0.042845
=,=,=,=,+,sp,a,r,-,b,u,k,J,J      0.024472
+,h,I,N,-,k,@,l,-,bl,O,k,J,J      0.147489
-,m,e,=-,d,A,l,+,j,O,n,E,E      0.182421
-,sn,u,=-,p,@,=,+,r,K,=,T,T      0.046229
=,=,=,=,=,=,=,+,sp,A,N,E,E      0.042845
+,k,a,=-,k,@,=-,n,E,st,J,J      0.114685
```

This can be used to study how very similar instances (low distance) and less similar patterns (higher distance) are used in the process of generalization.

The listing of nearest neighbors is useful for the analysis of the behavior of a classifier. It can be used to interpret why particular decisions or errors occur.

```
> timbl +v n+k -mM -k3 -w0 -f dimin.train -t dimin.test
```

```

+,m,I,=-,d,A,G,-,d,},t,J,J { J 3.00000 }
# k=1, 1 Neighbor(s) at distance: 0.99179269134432
# +,p,a,=-,t,@,rs,-,f,A,t,{ J 1.00000 }
# k=2, 1 Neighbor(s) at distance: 0.99458957262696
# +,h,o,=-,n,@,G,-,b,A,k,{ J 1.00000 }
# k=3, 1 Neighbor(s) at distance: 1.0088291749842
# +,h,E,r,-,d,@,rs,-,t,A,s,{ J 1.00000 }
-,t,@,=-,l,|,=-,G,@,n,T,T { T 3.00000 }
# k=1, 1 Neighbor(s) at distance: 0.33024081383366
# -,x,@,=-,h,|,=-,G,@,n,{ T 1.00000 }
# k=2, 1 Neighbor(s) at distance: 0.49144604610567
# -,d,@,r,-,w,a,=-,G,@,n,{ T 1.00000 }
# k=3, 1 Neighbor(s) at distance: 0.56944572926932
# -,st,@,=-,l,I,=-,N,@,=-,{ T 1.00000 }

```

A confusion matrix, printed when the `+v cm` option is selected, can bring to light specific errors of the classifier that would not be apparent from the overall accuracy. Applied to the diminutive data, the following confusion matrix is computed and printed:

```
> timbl +v cm -f dimin.train -t dimin.test
```

Confusion Matrix:

	T	E	J	P	K
T	453	0	2	0	0
E	0	87	4	1	8
J	1	5	346	0	0
P	0	3	0	24	0
K	0	7	0	0	9
--	0	0	0	0	0

The confusion matrix associates the class predicted by TiMBL (vertically) with the real class of the test items given (horizontally). All cells outside the diagonal contain errors of one class being mistaken for another. For example, the K class (*-kje*) is mispredicted seven times as class E (*-etje*).

(The bottom line, labeled with `--`, would contain aggregate counts of classes occurring in the test data that did not occur in the training data. In the diminutive data this does not occur.)

In general, a confusion matrix allows a more fine-grained analysis of experimental results and better experimental designs (some parameter settings may work for some classes but not for others, or some may improve recall, and others precision, e.g.). From such a matrix, not only accuracy can be derived, but also a number of additional metrics that have become popular in machine learning, information retrieval, and subsequently also in computational linguistics: *recall*, *precision*, and their harmonic mean *F-score*, as well as *true positive rate*, *false positive rate*, and their joint measure *AUC* in ROC space. The details of these advanced statistics are given in Section 5.6.

They can be reported by TiMBL using the `+v as` and `+v cs` verbosity options:

```
> timbl +v as+cs -f dimin.train -t dimin.test
```

Scores per Value Class:

class	TP	FP	TN	FN	precision	recall (TPR)	FPR	F-score
T	453	1	494	2	0.99780	0.99560	0.00202	0.99670
E	87	15	835	13	0.85294	0.87000	0.01765	0.86139
J	346	6	592	6	0.98295	0.98295	0.01003	0.98295
P	24	1	922	3	0.96000	0.88889	0.00108	0.92308
K	9	8	926	7	0.52941	0.56250	0.00857	0.54545

F-Score beta=1, microav: 0.967160

```
F-Score beta=1, macroav: 0.861914
AUC, microav:           0.980138
AUC, macroav:           0.926060
overall accuracy:       0.967368 (919/950), of which 39 exact matches
There were 5 ties of which 4 (80.00%) were correctly resolved
```

We hope that this tutorial has made it clear that, once you have coded your data in fixed-length feature-value patterns, it should be relatively straightforward to get the first results using TiMBL. You can then experiment with different metrics and algorithms to try and further improve your results.

Chapter 5

Memory-based learning algorithms

TiMBL is a program implementing several memory-based learning algorithms. All implemented algorithms have in common that they store some representation of the training set explicitly in memory. During testing, new cases are classified by extrapolation from the most similar stored cases. The main differences among the algorithms incorporated in TiMBL lie in:

- The definition of *similarity*,
- The way the instances are stored in memory, and
- The way the search through memory is conducted.

In this chapter, various choices for these issues are described. We start in Section 5.1 with a formal description of the basic memory-based learning algorithm, i.e. a nearest neighbor search. We then introduce different distance metrics, such as Information Gain weighting, which allows us to deal with features of differing importance, and the Modified Value Difference metric, which allows us to make a graded guess of the match between two different symbolic values, and describe the standard versus three distance-weighted versions of the class voting mechanism of the nearest neighbor classifier. In Section 5.2, we give a description of various algorithmic optimizations for nearest neighbor search.

Sections 5.3 to 5.5 describe three variants of the standard nearest neighbor classifier implemented within TiMBL, that optimize some intrinsic property of the standard algorithm. First, in Section 5.3, we describe IGTREE, which replaces the exact nearest neighbor search with a very fast heuristic that exploits the difference in importance between features. Second, in Section 5.4, we describe the TRIBL algorithm, which is a hybrid between IGTREE and nearest neighbor search. Third, Section 5.5 describes the IB2 algorithm, which incrementally and selectively adds instances to memory during learning.

The chapter is concluded by Section 5.7, which provides an overview of further reading into theory and applications of memory-based learning to natural language processing tasks.

5.1 Memory-based learning

Memory-based learning is founded on the hypothesis that performance in cognitive tasks is based on reasoning on the basis of similarity of new situations to *stored representations of ear-*

lier experiences, rather than on the application of *mental rules* abstracted from earlier experiences (as in rule induction and rule-based processing). The approach has surfaced in different contexts using a variety of alternative names such as similarity-based, example-based, exemplar-based, analogical, case-based, instance-based, and lazy learning (Stanfill and Waltz, 1986; Aha, Kibler, and Albert, 1991; Cost and Salzberg, 1993; Kolodner, 1993; Aha, 1997). Historically, memory-based learning algorithms are descendants of the k -nearest neighbor (henceforth k -NN) algorithm (Cover and Hart, 1967; De vij ver and Kittler, 1982; Aha, Kibler, and Albert, 1991).

An MBL system, visualized schematically in Figure 5.1, contains two components: a *learning component* which is memory-based (from which MBL borrows its name), and a *performance component* which is similarity-based.

The learning component of MBL is memory-based as it involves adding training instances to memory (the *instance base* or case base); it is sometimes referred to as ‘lazy’ as memory storage is done without abstraction or restructuring. An instance consists of a fixed-length vector of n feature-value pairs, and an information field containing the classification of that particular feature-value vector.

In the performance component of an MBL system, the product of the learning component is used as a basis for mapping input to output; this usually takes the form of performing classification. During classification, a previously unseen test example is presented to the system. The similarity between the new instance X and all examples Y in memory is computed using some *distance metric* $\Delta(X, Y)$. The extrapolation is done by assigning the most frequent category within the found set of most similar example(s) (the k -nearest neighbors) as the category of the new test example. In case of a tie among categories, a tie breaking resolution method is used. This method is described in subsection 5.1.7.

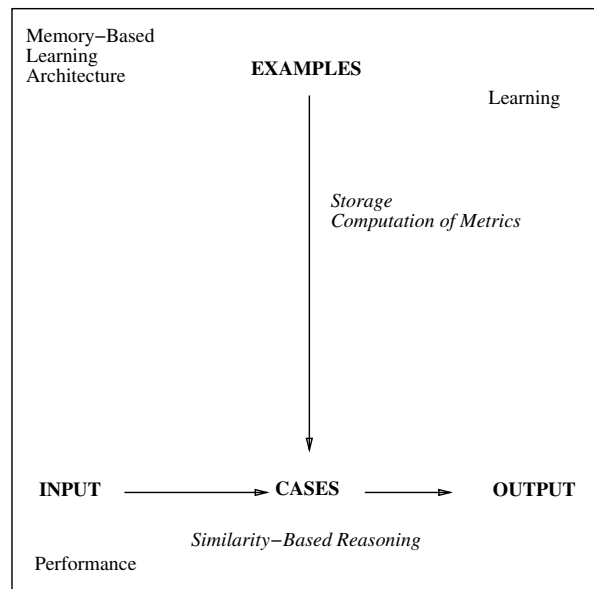


Figure 5.1: General architecture of an MBL system.

5.1.1 The Overlap metric

The most basic metric that works for patterns with symbolic features is the **Overlap metric**¹ given in Equations 5.1 and 5.2; where $\Delta(X, Y)$ is the distance between instances X and Y , represented by n features, and δ is the distance per feature. The distance between two patterns is simply the sum of the differences between the features. The k -NN algorithm with this metric is called IB1 (Aha, Kibler, and Albert, 1991).

$$\Delta(X, Y) = \sum_{i=1}^n \delta(x_i, y_i) \quad (5.1)$$

where:

$$\delta(x_i, y_i) = \begin{cases} \text{abs}(\frac{x_i - y_i}{\max_i - \min_i}) & \text{if numeric, else} \\ 0 & \text{if } x_i = y_i \\ 1 & \text{if } x_i \neq y_i \end{cases} \quad (5.2)$$

The major difference with the IB1 algorithm originally proposed by (Aha, Kibler, and Albert, 1991), is that in our version the value of k refers to k -nearest *distances* rather than k -nearest examples. With $k = 1$, for instance, TiMBL's nearest neighbor set can contain several instances that are equally distant to the test instance. Arguably, our k -NN kernel could therefore be called k -nearest distances classification.

Another difference with the original IB1 as well as with other implementations such as k -NN in the WEKA machine learning toolkit (Witten and Frank, 1999) is the way in which ties are resolved in choosing the majority category among the set of nearest neighbors. Since this method is independent of the distance function we discuss this issue separately in subsection 5.1.7.

Variations on Overlap: Levenshtein and Dice coefficient metrics The Overlap metric is all-or-nothing. For measuring the similarity between numeric or atomically symbolic values this may suffice, but there are cases (such as in natural language processing) in which string-valued feature values occur that can mismatch with other string values in a meaningfully graded way. For example, the value pair “bathe” and “bathes” only differs in one letter; counting them as more similar than “bathe” and “rumour”, for example, may be useful for the classification task at hand. We implemented two additional metrics, Levenshtein distance and the Dice coefficient, that each provide a graded similarity score between pairs of strings.

Levenshtein distance is a classic *edit distance* metric (Levenshtein, 1966) that counts the number of insertions, deletions, and substitutions to transform the one string into the other. In our (dynamic programming) implementation the three operations count equally heavily. The **Dice** coefficient computes the overlap between the occurrences of character bigrams in two strings as in Equation 5.3, where $n_{x_i \cap y_i}$ is the number of character bigrams (uniquely) occurring both in string value x_i and in string value y_i (and where i is the index of the feature as introduced in Equation 5.1)². The equation subtracts the similarity from 1, because we assume δ to produce a distance, not a similarity.

$$\delta(x_i, y_i) = 1 - \frac{2n_{x_i \cap y_i}}{n_{x_i} + n_{y_i}} \quad (5.3)$$

¹This metric is also referred to as Hamming distance, Manhattan metric, city-block distance, or L1 metric.

²Strings of length one are not handled by Dice; we back off to Overlap in these cases.

5.1.2 Information-gain and gain ratio feature weighting

The distance metric in Equation 5.2 straightforwardly counts the number of (mis)matching feature-values in both patterns. In the absence of information about feature relevance, this is a reasonable choice. Otherwise, we can add domain knowledge bias to weight or select different features (see e.g. Cardie (1996) for an application of linguistic bias in a language processing task), or look at the behavior of features in the set of examples used for training. We can compute statistics about the relevance of features by looking at which features are good predictors of the class labels. Information Theory gives us a useful tool for measuring feature relevance in this way (Quinlan, 1986; Quinlan, 1993).

Information Gain (IG) weighting looks at each feature in isolation, and measures how much information it contributes to our knowledge of the correct class label. The Information Gain of feature i is measured by computing the difference in uncertainty (i.e. entropy) between the situations without and with knowledge of the value of that feature (Equation 5.4).

$$w_i = H(C) - \sum_{v \in V_i} P(v) \times H(C|v) \quad (5.4)$$

Where C is the set of class labels, $H(C) = -\sum_{c \in C} P(c) \log_2 P(c)$ is the entropy of the class labels, V_i is the set of values for feature i , and $H(C|v)$ is the conditional entropy of the subset of the training examples that have value v on feature i . The probabilities are estimated from relative frequencies in the training set.

For numeric features, an intermediate step needs to be taken to apply the symbol-based computation of IG. All real values of a numeric features are temporarily discretized into a number (the default is 20) of intervals. Instances are ranked on their real value, and then spread evenly over the intervals; each interval contains the same number of instances (i.e., by default, 1/20th of the total amount of instances). Instances in each of these intervals are then used in the IG computation as all having the same unordered, symbolic value per group. Note again that this discretization is only temporary; it is not used in the computation of the distance metric.

It is important to realize that the IG weight is really a probability-weighted average of the informativity of the different values of the feature. On the one hand, this pre-empts the consideration of values with low frequency but high informativity. Such values “disappear” in the average. On the other hand, this also makes the IG weight very robust to estimation problems. Each parameter (weight) is estimated on the whole data set.

Information Gain, however, tends to overestimate the relevance of features with large numbers of values. Imagine a data set of hospital patients, where one of the available features is a unique “patient ID number”. This feature will have very high Information Gain, but it does not give any generalization to new instances. To normalize Information Gain for features with different numbers of values, Quinlan (Quinlan, 1993) has introduced a normalized version, called **Gain Ratio**, which is Information Gain divided by $si(i)$ (split info), the entropy of the feature-values (Equation 5.6).

$$w_i = \frac{H(C) - \sum_{v \in V_i} P(v) \times H(C|v)}{si(i)} \quad (5.5)$$

$$si(i) = - \sum_{v \in V_i} P(v) \log_2 P(v) \quad (5.6)$$

The resulting Gain Ratio values can then be used as weights w_f in the weighted distance metric (Equation 5.7)³. The k -NN algorithm with this metric is called IB1-IG (Daelemans and Van den Bosch, 1992).

$$\Delta(X, Y) = \sum_{i=1}^n w_i \delta(x_i, y_i) \quad (5.7)$$

The possibility of automatically determining the relevance of features implies that many different and possibly irrelevant features can be added to the feature set. This is a very convenient methodology if domain knowledge does not constrain the choice enough beforehand, or if we wish to measure the importance of various information sources experimentally. However, because IG values are computed for each feature independently, this is not necessarily the best strategy. Sometimes better results can be obtained by leaving features out than by letting them in with a low weight. Very redundant features can also be challenging for IB1-IG, because IG will overestimate their joint relevance. Imagine an informative feature which is duplicated. This results in an overestimation of IG weight by a factor two, and can lead to accuracy loss, because the doubled feature will dominate the distance metric.

5.1.3 Chi-squared and shared variance feature weighting

Unfortunately, as White and Liu (1994) have shown, the Gain Ratio measure still has an unwanted bias towards features with more values. The reason for this is that the Gain Ratio statistic is not corrected for the number of degrees of freedom of the contingency table of classes and values. White and Liu (1994) proposed a feature selection measure based on the chi-squared statistic, as values of this statistic can be compared across conditions with different numbers of degrees of freedom.

The χ^2 statistic is computed from the same contingency table as the Information Gain measure by the following formula (Equation 5.8).

$$\chi^2 = \sum_i \sum_j \frac{(E_{ij} - O_{ij})^2}{E_{ij}} \quad (5.8)$$

where O_{ij} is the observed number of cases with value v_i in class c_j , i.e. $O_{ij} = n_{ij}$, and E_{ij} is the expected number of cases which should be in cell (v_i, c_j) in the contingency table, if the null hypothesis (of no predictive association between feature and class) is true (Equation 5.9). Let $n_{.j}$ denote the marginal for class j (i.e. the sum over column j of the table), $n_{i.}$ the marginal for value i , and $n_{..}$ the total number of cases (i.e. the sum of all the cells of the contingency table).

$$E_{ij} = \frac{n_{.j}n_{i.}}{n_{..}} \quad (5.9)$$

The χ^2 statistic is well approximated by the chi-square distribution with $\nu = (m-1)(n-1)$ degrees of freedom, where m is the number of values and n is the number of classes. We can then either use the χ^2 values as feature weights in Equation 5.7, or we can explicitly correct for the degrees of freedom by using the **Shared Variance** measure (Equation 5.10).

³In a generic use IG refers both to Information Gain and to Gain Ratio throughout this manual. In specifying parameters for the software, the distinction between both needs to be made, because they often result in different behavior.

$$SV_i = \frac{\chi_i^2}{N \times (\min(|C|, |V_i|) - 1)} \quad (5.10)$$

Where $|C|$ and $|V_i|$ are the number of classes and the number of values of feature i , respectively, and N is the number of instances⁴. We will refer to these variations of MBL as IB1- χ^2 and IB1-SV.

One should keep in mind, that the correspondence to the chi-square distribution generally becomes poor if the expected frequencies in the contingency table cells become small. A common recommendation is that the χ^2 test cannot be trusted when more than 20% of the expected frequencies are less than 5, or any are less than 1.

Chi-squared and shared variance weights of *numeric* features are computed via a discretization preprocessing step (also used with computing IG and GR weights). Values are first discretized into a number (20 by default) of equally-spaced intervals between the minimum and maximum values of the feature. These groups are then used as discrete values in computing chi-squared and shared variance weights.

5.1.4 Modified value difference, Jensen-Shannon divergence, and Jeffrey divergence metrics

The choice of representation for instances in MBL remains the key factor determining the strength of the approach. The features and categories in NLP tasks are usually represented by symbolic labels. The metrics that have been described so far, i.e. Overlap and IG Overlap, are limited to counting exact matches between feature values. This means that all values of a feature are seen as equally dissimilar. However, if we think of an imaginary task in e.g. the phonetic domain, we might want to use the information that 'b' and 'p' are more similar than 'b' and 'a'. For this purpose a metric was defined by Stanfill and Waltz (1986) and further refined by Cost and Salzberg (1993). It is called the (Modified) Value Difference Metric (MVDM; Equation 5.11), and it is a method to determine the similarity of the values of a feature by looking at co-occurrence of values with target classes. For the distance between two values v_1, v_2 of a feature, we compute the difference of the conditional distribution of the classes C_i for these values.

$$\delta(v_1, v_2) = \sum_{i=1}^n |P(C_i|v_1) - P(C_i|v_2)| \quad (5.11)$$

For computational efficiency, all pairwise $\delta(v_1, v_2)$ values can be precomputed before the actual nearest neighbor search starts.

Although the MVDM metric does not explicitly compute feature relevance, an implicit feature weighting effect is present. If features are very informative, their conditional class probabilities will often be skewed towards a particular class. This implies that on average the $\delta(v_1, v_2)$ will be large. For uninformative features, on the other hand, the conditional class probabilities will tend to be closer to the overall class distribution, so that on average the $\delta(v_1, v_2)$ will be very small.

MVDM differs considerably from Overlap based metrics in its composition of the nearest neighbor sets. Overlap causes an abundance of ties in nearest neighbor position. For example, if the nearest neighbor is at a distance of one mismatch from the test instance, then the nearest neighbor set will contain the entire partition of the training set that matches all the other features but contains

⁴Note that with two classes, the shared variance weights of all features are simply divided by N , and will not be different from χ^2 weights.

any value for the mismatching feature (see Zavrel and Daelemans (1997) for a more detailed discussion). With the MVDM metric, however, the nearest neighbor set will either contain patterns which have the value with the lowest $\delta(v_1, v_2)$ in the mismatching position, or MVDM will select a totally different nearest neighbor which has less exactly matching features, but a smaller distance in the mismatching features. In sum, this means that the nearest neighbor set is usually much smaller for MVDM at the same value of k . In NLP tasks we have found it useful to experiment with values of k larger than one for MVDM, because this re-introduces some of the beneficial smoothing effects associated with large nearest neighbor sets.

One cautionary note about this metric is connected with data sparsity. In many practical applications we are confronted with a very limited set of examples, with values occurring only a few times or once in the whole data set. If two such values occur with the same class, MVDM will regard them as identical, and if they occur with two different classes their distance will be maximal. In cases of such extreme behaviour on the basis of low-frequency evidence, it may be safer to back off to the Overlap metric, where only an exact value match yields zero distance. TiMBL offers this back-off from MVDM to Overlap through a frequency threshold, that switches from the MVDM to the Overlap metric when one or both of a pair of matched values occurs fewer times in the learning material than this threshold.

Jensen-Shannon divergence and Jeffrey divergence are offered as related, but more complex alternatives to MVDM. They are both statistical dissimilarity metrics that can be used to compute the distance between class distributions of two values of the same feature. Functionally they are quite similar to MVDM as well as to Kullback-Leibler (KL) divergence; Jeffrey divergence and Jensen-Shannon divergence are both symmetric versions of KL divergence, and symmetry is what we need for estimating value difference.

First, Jeffrey divergence is best known for its application as a distance function in unsupervised vector space models, e.g. in image retrieval, where it is applied to histogram vectors. While MVDM computes a straightforward geometrical distance between two class distribution vectors, Jeffrey divergence introduces a logarithm term, as seen in Equation 5.12. Jeffrey divergence is a symmetric variant of Kullback-Leibner distance; the m term given in Equation 5.13 is used for this purpose.

$$\delta(v_1, v_2) = \sum_{i=1}^n (P(C_i|v_1) \log \frac{P(C_i|v_1)}{m} + P(C_i|v_2) \log \frac{P(C_i|v_2)}{m}) \quad (5.12)$$

$$m = \frac{P(C_i|v_1) + P(C_i|v_2)}{2} \quad (5.13)$$

Compared to MVDM, Jeffrey divergence assigns relatively larger distances to value pairs of which the class distributions are more orthogonal. In other words, it assigns more prominence to zero probabilities, which in the case of sparse data (e.g. with Zipfian distributions of values) are generally better estimations than non-zero probabilities. This makes Jeffrey divergence in principle more robust than MVDM with respect to sparse data.

As with MVDM, TiMBL offers an optional frequency-thresholded back-off from Jeffrey and Jensen-Shannon divergence to the Overlap metric to further remedy some negative effects due to data sparseness.

5.1.5 Dot-product and cosine metrics

When features have numeric or binary values, TiMBL can also compute the distance between two instances via the dot product (or inner product) of their feature-value vectors. The dot product (which is higher with better matches) is subsequently inversed to a distance by subtracting it from the maximum dot product attainable, i.e. that on an exact match. In Equation 5.14 this maximal dot product is referred to as dot_{max} .

$$\Delta(X, Y) = dot_{max} - \sum_{i=1}^n w_i x_i y_i \quad (5.14)$$

As with the other distance metrics incorporated in TiMBL, we include the feature weight w_i in the metric. When no weighting is set ($-w \ 0$), all weights are set to 1.0, and equation 5.14 reduces to the normal unweighted dot product.

The dot-product metric is typically used with binary vectors or sparse vectors in general. When either x_i or y_i has a zero value, that value pair is not counted in the dot product. Consequently, the significant deviation from the Overlap metric is that matching values that both have a zero value do not count here, whereas they count as much as any other value match in the Overlap metric.

A commonly used variant of the dot product metric, e.g. in information retrieval, is the cosine metric, which corrects for large differences in the length of the instance vectors. The cosine metric divides the dot product metric by the product of the length of the two vectors. As with the dot product, TiMBL converts the cosine metric similarity to a distance by subtracting it from a cos_{max} term that is larger than the maximal cosine similarity, as given in Equation 5.15. Again, feature weighting is included in the formula:

$$\Delta(X, Y) = cos_{max} - \frac{\sum_{i=1}^n w_i x_i y_i}{\sqrt{\sum_{i=1}^n w_i x_i^2 \sum_{i=1}^n w_i y_i^2}} \quad (5.15)$$

Due to its internal tree structure, TiMBL is not particularly suited to handle feature vectors with thousands or more features. Many features cause very deep and usually very unbalanced trees, from which retrieval can be rather inefficient (especially when there is little variance in the feature weights). Other internal data structures such as inverted indices are typically more suited to these types of vector spaces. For now, inverted indices are not implemented in TiMBL.

5.1.6 Distance-weighted class voting

The most straightforward method for letting the k nearest neighbors vote on the class of a new case is the *majority voting* method, in which the vote of each neighbor receives equal weight, and the class with the highest number of votes is chosen (or in case of a tie, some tie resolution is performed, cf. Subsection 5.1.7).

We can see the voting process of the k -NN classifier as an attempt to make an optimal class decision, given an estimate of the conditional class probabilities in a local region of the data space. The radius of this region is determined by the distance of the k -furthest neighbor.

Sometimes, if k is small, and the data is very sparse, or the class labels are noisy, the “local” estimate is very unreliable. As it turns out in experimental work, using a larger value of k can often lead to higher accuracy. The reason for this is that in densely populated regions, with larger k the

local estimates become more reliable, because they are "smoother". However, when the majority voting method is used, smoothing can easily become oversmoothing in sparser regions of the same data set. The reason for this is that the radius of the k -NN region can become extended far beyond the local neighborhood of the query point, but the far neighbors will receive equal influence as the close neighbors. This can result in classification errors that could easily have been avoided if the measure of influence would somehow be correlated with the measure of similarity. To remedy this, we have implemented three types of distance weighted voting functions in TiMBL.

A voting rule in which the votes of different members of the nearest neighbor set are weighted by a function of their distance to the query, was first proposed by Dudani (1976). In this scheme, henceforth referred to as IL (for inverse-linear), a neighbor with smaller distance is weighted more heavily than one with a greater distance: the nearest neighbor gets a weight of 1, the furthest neighbor a weight of 0 and the other weights are scaled linearly to the interval in between (Dudani (1976), Equation 5.16.).

$$w_j = \begin{cases} \frac{d_k - d_j}{d_k - d_1} & \text{if } d_k \neq d_1 \\ 1 & \text{if } d_k = d_1 \end{cases} \quad (5.16)$$

Where d_j is the distance to the query of the j 'th nearest neighbor, d_1 the distance of the nearest neighbor, and d_k of the furthest (k 'th) neighbor.

Dudani (Dudani (1976), eq. 2 and 3) further proposed the *inverse distance weight* (henceforth ID). In Equation 5.17 a small constant is usually added to the denominator to avoid division by zero (Wettschereck, 1994).

$$w_j = \left\{ \frac{1}{d_j + \epsilon} \right. \quad (5.17)$$

Another weighting function considered here is based on the work of Shepard (1987), who argues for a universal perceptual law which states that the relevance of a previous stimulus for the generalization to a new stimulus is an exponentially decreasing function of its distance in a psychological space (henceforth ED). This gives the weighed voting function of Equation 5.18, where α and β are constants determining the slope and the power of the exponential decay function.

$$w_j = e^{-\alpha d_j^\beta} \quad (5.18)$$

Note that in Equations 5.17 and 5.18 the weight of the nearest and furthest neighbors and the slope between them depend on their absolute distance to the query. This assumes that the relationship between absolute distance and the relevance gradient is fixed over different datasets. This assumption is generally false; even within the same dataset, different feature weighting metrics can cause very different absolute distances.

Figure 5.2 visualises a part of the curves of ID and ED, the latter with a few varied settings of α and β . Generally, both distance weighting functions assign highly differing weights for close neighbors, and less differing weights for more distant neighbors. ID assigns very high votes (distance weights) to nearest neighbors at distances approaching 0.0 - in effect it assigns absolute preference to exact matches. In contrast, all ED variants have a vote of 1.0 for exact matches, and have a shallower curve than the ID curve for higher distances. Higher values of α in the ED function assign relatively higher weights to exact matches. When β is set to larger values than 1.0, the ED curve becomes bell-shaped, effectively assigning relatively less different weights between exact-matching neighbors and near-exact matching instances.

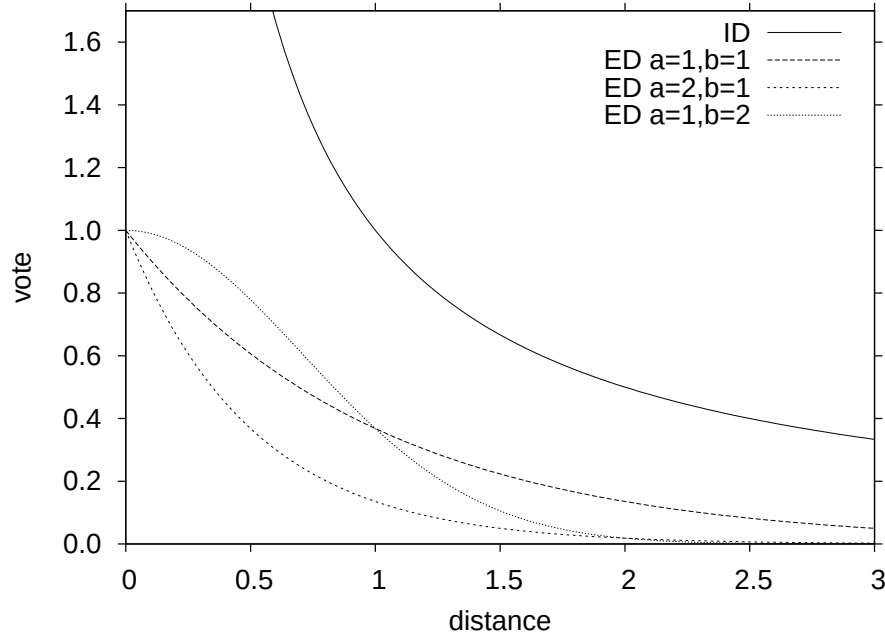


Figure 5.2: Visualisation of the Inverse Distance weighting function (ID) and three variants of the Exponential Decay distance weighting function (ED) varying settings of α (1) and β (b).

Following Dudani's proposal, the benefits of weighted voting for k -NN have been discussed widely, e.g. (Bailey and Jain, 1978; Morin and Raeside, 1981; MacLeod, Luk, and Titterington, 1987), but mostly from an analytical perspective. With the popularity of Instance-Based Learning applications, these issues have gained a more practical importance. In his thesis on k -NN classifiers, Wettschereck (1994) cites Dudani, but proceeds to work with Equation 5.17. He tested this function on a large amount of datasets and found weak evidence for performance increase over majority voting. An empirical comparison of the discussed weighted voting methods in (Zavrel, 1997) has shown that weighted voting indeed often outperforms unweighted voting, and that Dudani's original method (Equation 5.16) mostly outperforms the other two methods. From that set of experiments, it also seems that Dudani's method shows its optimal performance at much larger values of k than the other voting methods.

5.1.7 Tie breaking

Thus far we have described the last step of k -NN classification as taking the majority category among the set of nearest neighbors, where their vote is either unweighted or weighted by their distance (subsection 5.1.6). Especially in case of unweighted voting, ties can occur; e.g. of a set of ten nearest neighbors, five vote for class A , and the other five for B . The procedure for breaking this tie in the k -NN classifier in TiMBL is as follows. First, the value of the k parameter is incremented by 1, and the additional nearest neighbors at this new k th distance are added to the current nearest neighbor set (k is subsequently reset to its user-specified value). If the tie in the class distribution persists, then the class label is selected with the highest overall occurrence in the training set. If that is also equal, then the first class is taken that was encountered when reading the training instance file.

Optionally, TiMBL can be set to avoid ties by making a *random* choice of a classification from a class distribution in a nearest-neighbor set, weighted by the distribution of the classes in the set.

5.1.8 Exemplar weighting

Exemplar weighting in memory-based learning captures the intuition that some instances are better (more reliable, more typical, more regular) nearest neighbors than others. Classification accuracy could benefit from giving these instances some priority in the k -NN classification process. This idea has been explored in the context of on the one hand classification (Salzberg, 1990; Zhang, 1992), and on the other hand editing bad instances from memory (Aha, Kibler, and Albert, 1991). Salzberg (1990), as a classic example, uses *class-prediction strength*: the ratio of the number of times the instance type is a nearest neighbor of another instance with the same class and the number of times that the instance type is the nearest neighbor of another instance type regardless of the class. Another example is *typicality* as used by Zhang (1992).

Exemplar weights could in principle be used either as weights in the class voting (as distance weights, cf. Subsection 5.1.6), or as weights in the distance metric (eq. 5.7). TiMBL supports only the latter type, and in this respect exemplar weighting is not an intrinsic part of TiMBL. TiMBL does not compute exemplar weighting metrics itself, but only allows users to specify preprocessed exemplar weights with the `-s` input option. Subsequently, when the distance between a test instance and a memory instance is computed, TiMBL uses the memory instance's weight as follows, where $\Delta^E(X, Y)$ is the exemplar-weighted distance between instances X and Y , and ew_X is the exemplar weight of memory instance X :

$$\Delta^E(X, Y) = \frac{\Delta(X, Y)}{ew_X + \epsilon} \quad (5.19)$$

ϵ is the smallest non-zero number, and is used to avoid division by zero. Exemplar weights approaching zero yield very large distances; relatively higher values yield relatively smaller distances.

Note that when a training instance occurs more than once in a training set, TiMBL expects it to have the same example weight with all occurrences; TiMBL cannot handle different example weights for the same instance type. TiMBL produces a warning (*Warning: deviating exemplar weight in line #. . .*), and uses the first weight found for the instance.

5.2 Indexing optimizations

The discussion of the algorithm and the metrics in the section above is based on a naive implementation of nearest neighbor search: a flat array of instances which is searched from beginning to end while computing the similarity of the test instance with each training instance. Such an implementation, unfortunately, reveals the flip side of the lazy learning coin. Although learning is very cheap: just storing the instances in memory, the computational price of classification can become very high for large data sets. The computational cost is proportional to N , the number of instances in the training set, times f , the number of features. In our current implementation of IB1 we use tree-based indexing to alleviate these costs.

5.2.1 Tree-based indexing

The tree-based memory indexing operation replaces the flat array by a tree structure. Instances are stored in the tree as paths from a root node to a leaf, the arcs of the path are the consecutive feature-values, and the leaf node contains a *distribution* of classes, i.e. a count of how many times

which class occurs with this pattern of feature-values. Due to this storage structure, instances with identical feature-values are collapsed into a single path, and only their separate class information needs to be stored in the distribution at the leaf node. Many different **tokens** of a particular **instance type** share one path from the root to a leaf node. Moreover, instances which share a prefix of feature-values, also share a partial path. This reduces storage space (although at the cost of some book-keeping overhead) and has two implications for nearest neighbor search efficiency.

First, the tree can be searched top-down very quickly for *exact matches*. When $k = 1$, an exact match ($\Delta(X, Y) = 0$) can never be beaten, so then it is possible to omit any further distance computations. The shortcut is built into TiMBL, but by default it is not used with $k > 1$. TiMBL does, however, offer the possibility to use the shortcut at any value of k , with the command line switch (+x. Using it can speed up classification radically for some types of data, but with $k > 1$, the shortcut is not guaranteed to give the same performance (for better or for worse) as classification without it.

Second, the distance computation for the nearest neighbor search can re-use partial results for paths which share prefixes. This re-use of partial results is in the direction from the root to the leaves of the tree. When we have proceeded to a certain level of the tree, we know how much similarity (Equation 5.2) can still contribute to the overall distance (Equation 5.1), and discard whole branches of the tree which will never be able to rise above the partial similarity of the current least similar nearest neighbor. By doing the search depth first⁵, the similarity threshold quickly gets initialized to a good value, so that large parts of the search space can be pruned⁶.

Disregarding this last constraint on search, the number of feature-value comparisons is equal to the number of arcs in the tree. Thus if we can find an ordering of the features which produces more overlap between partial paths, and hence a smaller tree, we can gain both space and time improvements. An ordering which was found to produce small trees for many of our NLP data sets is Gain Ratio divided by the number of feature-values (this is the default setting). Through the `--Treeorder=<string>` command line switch, however, the user is allowed to experiment with different orderings. Note that different orderings may only affect classification speed, not the actual classifications.

5.3 IGTREE

Using Information Gain rather than unweighted Overlap distance to define similarity in IB1 improves its performance on several NLP tasks (Daelemans and Van den Bosch, 1992; Van den Bosch and Daelemans, 1993; Van den Bosch, 1997). The positive effect of Information Gain on performance prompted us to develop an alternative approach in which the instance memory is restructured in such a way that it contains the same information as before, but in a compressed decision tree structure. We call this algorithm IGTREE (Daelemans, Van den Bosch, and Weijters, 1997) In this structure, similar to the tree-structured instance base described above, instances are stored as paths of connected nodes which contain classification information. Nodes are connected via arcs denoting feature values. Information Gain is used to determine the order in which instance feature-values are added as arcs to the tree. The reasoning behind this compression is that when the computation of information gain points to one feature clearly being the most important in classification, search can be restricted to matching a test instance to those memory instances that have the same feature-value as the test instance at that feature. Instead of indexing all memory instances only once on this feature, the instance memory can then be optimized further by ex-

⁵Suggested by Gert Durieux.

⁶With the special command line setting `--silly=true` this tree search shortcut is switched off; as the name of the setting suggests, this is not recommended, except for explicit speed comparisons.

amining the second most important feature, followed by the third most important feature, etc. Again, compression is obtained as similar instances share partial paths.

Because IGTREE makes a heuristic approximation of nearest neighbor search by a top down traversal of the tree in the order of feature relevance, we no longer need to store all the paths. The idea is that it is not necessary to fully store those feature-values of the instance that have lower Information Gain than those features which already fully disambiguate the instance classification.

Apart from compressing all training instances in the tree structure, the IGTREE algorithm also stores with each non-terminal node information concerning the *most probable* or *default* classification given the path thus far, according to the bookkeeping information maintained by the tree construction algorithm. This extra information is essential when processing unknown test instances. Processing an unknown input involves traversing the tree (i.e., matching all feature-values of the test instance with arcs in the order of the overall feature Information Gain), and either retrieving a classification when a leaf is reached (i.e., an exact match was found), or using the default classification on the last matching non-terminal node if an exact match fails.

In sum, it can be said that in the trade-off between computation during learning and computation during classification, the IGTREE approach chooses to invest more time in organizing the instance base using Information Gain and compression, to obtain simplified and faster processing during classification, as compared to IB1 and IB1-IG.

The generalization accuracy of IGTREE is usually comparable to that of IB1-IG; often slightly worse, but sometimes even better. The two causes for IGTREE's surprisingly good accuracies attained with dramatically faster classification are that (i) most 'unseen' instances contain large parts that fully match stored parts of training instances, and (ii) the probabilistic information stored at non-terminal nodes (i.e., the default classifications) still produces strong 'best guesses' when exact matching fails. The difference between the top-down traversal of the tree and precise nearest neighbor search becomes more pronounced when the differences in informativity between features are small. In such a case a slightly different weighting would have produced a switch in the ordering and a completely different tree. The result can be a considerable change in classification outcomes, and hence also in accuracy. However, we have found in our work on NLP datasets that when the goal is to obtain a very fast classifier for processing large amounts of text, the tradeoff between a somewhat lower accuracy against stellar speed increases can be very attractive.

It should be noted that by design, IGTREE is not suited for numeric features, as it does not use some type of discretization. If present in data, numbers will simply be treated as literal strings by IGTREE. Moreover, one should realize that the success of IGTREE is determined by a good judgement of feature relevance ordering. Hence IGTREE is not to be used with e.g. "no weights" ($-w\ 0$). Also, setting the $-k$ parameter obviously has no effect on IGTREE performance.

5.4 The TRIBL and TRIBL2 hybrids

The application of IGTREE on a number of common machine-learning datasets suggested that it is not applicable to problems where the relevance of the predictive features cannot be ordered in a straightforward way, e.g. if the differences in Information Gain are only very small. In those cases, IB1-IG or even IB1 tend to perform significantly better than IGTREE.

For this reason we have designed TRIBL (Daelemans, Weijters, and Van den Bosch, 1997b) and TRIBL2 as hybrid combinations of IGTREE and IB1. Both algorithms aim to exploit the trade-off between (i) optimization of search speed (as in IGTREE), and (ii) maximal generalization ac-

curacy. They do that by splitting the classification of new instances into a quick decision-tree (IGTREE) traversal based on the first (most important and most class-disambiguating) features, followed by a slow but relatively accurate k -NN (IB1) classification based on the remaining less important features. The difference between TRIBL and TRIBL2 is that the former algorithm fixes the point in the feature ordering where IGTREE is succeeded by IB1, while TRIBL2 determines this switching point automatically per classification. We briefly describe both variants.

For TRIBL, a parameter $(-q)$ determines the switching point in the feature ordering from IGTREE to IB1. A heuristic that we have used with some success is based on *average feature information gain*; when the Information Gain of a feature exceeds the average Information Gain of all features + one standard deviation of the average, then the feature is used for constructing an IGTREE, including the computation of defaults on nodes. When the Information Gain of a feature is below this threshold, and the node is still ambiguous, tree construction halts and the leaf nodes at that point represent case bases containing subsets of the original training set. During search, the normal IGTREE search algorithm is used, until the case-base nodes are reached, in which case regular IB1 nearest neighbor search is used on this sub-case-base.

TRIBL2 does not employ a fixed switching point. Rather, during the classification of an instance it continues to use IGTREE as long as it finds matching feature values in the weighting-governed feature ordering. Only when it finds a mismatch it reverts to IB1 classification on all remaining features. The reasoning behind this mismatch-based switching is that it offers a fairly optimal minimalisation of the use of IB1; it is only invoked when mismatching occurs, which is the typical point in which IB1 can improve on decision-tree-style classification, which does not consider the other potentially matching features in the ordering (Daelemans, Van den Bosch, and Zavrel, 1999).

A final note: as with IGTREE, it does not make sense to use TRIBL and TRIBL2 without feature weighting, so do not combine TRIBL or TRIBL2 with $(-w \ 0)$.

5.5 IB2: Incremental editing

In memory-based learning it seems sensible to keep any instance in memory that plays a (potentially) positive role in the correct classification of other instances. Alternatively, when it plays no role at all, or when it is disruptive for classification, it may be a good idea to discard, or *edit* it from memory. On top of not harming or even improving generalization performance, the editing of instances from memory could also alleviate the practical processing burden of the k -NN classifier kernel, since it would have less instances to compare new instances to. This potential double pay-off spawned a distinct line of work on editing in the k -NN classifier quite early Hart (1968) and Wilson (1972).

TiMBL offers an implementation of one particular editing algorithm called IB2 (Aha, Kibler, and Albert, 1991), an extension to the basic IB1 algorithm introduced in the same article. IB2 implements an incremental editing strategy. Starting from a seed memory filled with a certain (usually small) number of labeled training instances, IB2 adds instances incrementally to memory only when they are *misclassified* by the k -NN classifier on the basis of the instances in memory at that point. These instances are added, since they are assumed to be representatives of a part of the complete instance space in which they themselves and potentially more nearest-neighbor instances have a particular class different from the class of neighboring instances already in memory. The economical idea behind IB2 is that this way typically only instances on the boundaries of such areas are stored, and not the insides of the areas; the classification of instances that would be positioned well inside such areas is assumed to be safeguarded by the memorized boundary instances surrounding it.

		true class	
		positive	negative
predicted class	correct	TP true positives	FP false positives
	incorrect	FN false negatives	TN true negatives
		P	N

Figure 5.3: Class-specific confusion matrix containing the basic counts used in the advanced performance metrics.

Although the IB2 may optimize storage considerably, its strategy to store all misclassified instances incrementally makes IB2 sensitive to noise (Aha, Kibler, and Albert, 1991). It is also yet unclear what the effect is of the size of the seed.

5.6 Advanced evaluation metrics

Aside from accuracy (the percentage of correctly classified test instances), TiMBL offers some more evaluation metrics that have become common in information retrieval and machine learning in general, namely precision, recall, and F-score, and ROC-space (with dimensions true positive rate and false positive rate), and AUC. We describe these metrics in more detail here.

Figure 5.3 displays the general confusion matrix for one class C , splitting all classifications on a test set into four cells. The TP or true positives cell contains a count of examples that have class C and are predicted to have this class correctly by the classifier. The FP or false positives cell contains a count of examples of a different class that the classifier incorrectly classified as C . The FN or false negatives cell contains examples of class C for which the classifier predicted a different class label than C . On the basis of these four numbers and the total number of positive examples $P = TP + FN$ and negative examples $N = FP + TN$, we can compute the following performance measures:

Precision = $\frac{TP}{TP+FP}$, or the proportional number of times the classifier has correctly made the decision that some instance has class C .

Recall or True Positive Rate (TPR) = $\frac{TP}{P}$, or the proportional number of times an example with class C in the test data has indeed been classified as class C by the classifier.

False Positive Rate (FPR) = $\frac{FP}{N}$, or the proportional number of times an example with a different class than C in the test data has been classified as class C by the classifier.

F-score = $\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$, or the harmonic mean of precision and recall (Van Rijsbergen, 1979), is a commonly used metric to summarize precision and recall in one measure. The left part

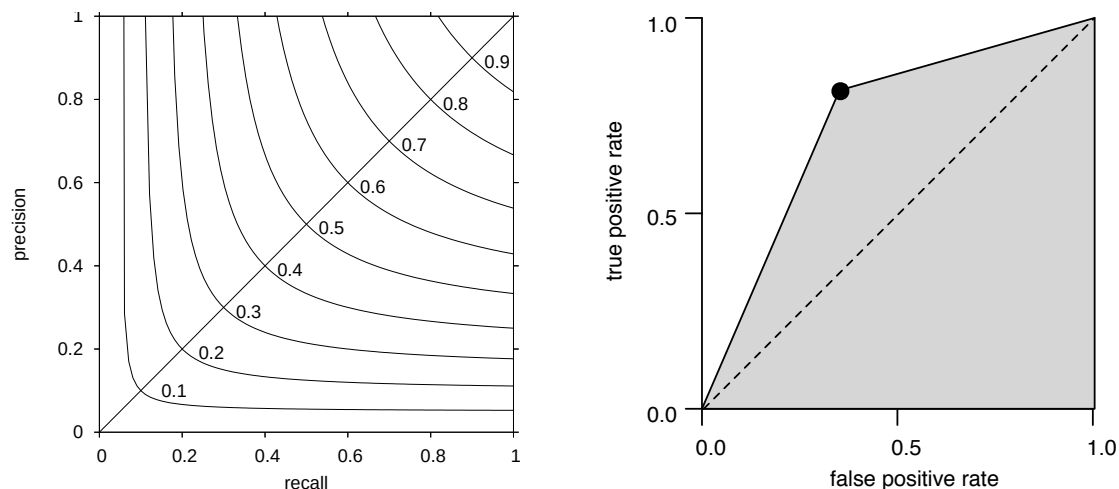


Figure 5.4: Precision–recall space with F-score isolines (left), and ROC space with an experimental outcome marked by the dot, and the outcome’s AUC, the shaded surface between the dot and coordinates (0, 0), (1, 0), and (1, 1) (right).

of Figure 5.4 shows F-score isolines in the two-dimensional space of recall (x-axis) and precision (y-axis). The curvature of the isolines is caused by the harmonic aspect of the formula (in contrast, the normal mean has straight isolines orthogonal to the $x = y$ diagonal), which penalizes large differences between precision and recall. The isolines could be likened to height isolines in a map, where the peak of the hill is at the upper right corner of the space.

AUC or *area under the curve* in the so-called ROC or *receiver operator characteristics* space (Egan, 1975; Swets, Dawes, and Monahan, 2000), is the surface of the grey area in the right graph of Figure 5.4. The ROC space is defined by the two dimensions FPR (false positive rate, x-axis) and TPR (true positive rate, or recall, y-axis). The difference with F-score is that it does not make use of the statistically unreliable precision metric; rather, it takes into account all cells of the matrix in Figure 5.3 including the TN (true negative) cell (for a more detailed description and arguments for using ROC analysis, cf. (Fawcett, 2004)). Its “peak” is in the upper left corner, at a FPR of zero and a TPR of 1. Rather than using the harmonic mean, it is common to report on the AUC, area under the classifier’s TPR-FPR curve, where in the case of a discrete-output classifier such as TiMBL this can be taken to mean the two lines connecting the experiment’s TPR and FPR to the (0, 0) coordinate and the (1, 1) coordinate, respectively; the AUC is then the grey area between these points and coordinate (1, 0).

While these advanced statistics can be computed per class, they can also be averaged to produce a single outcome for a full test set. Common methods for averaging F-scores and AUC scores are micro-averaging and macro-averaging. In micro-averaging, each class’ F-score or AUC is weighted proportionally to the frequency of the class in the test set. A macro-average sums the F-scores or AUCs and divides the total by the number of classes in the training set. In computing these averages, TiMBL bases itself on the classes in the training set. When a class does not re-occur in test material, it can have no recall, but it can have precision, hence it is always incorporated in averages. A class that occurs in test material but not in training material can never be predicted correctly, and is never included in averages.

5.7 Applications of TiMBL

This section provides a brief historical overview of work that has used TiMBL as a tool. Much of this work concerns tasks in natural language processing, but TiMBL has been applied in other domains as well. For historical background predating the 1990s, see (Daelemans and Van den Bosch, 2005).

Algorithmic development

As the original user and developer groups, the Tilburg and Antwerp groups have published a number of papers and articles containing descriptions of the algorithms and specialised metrics collected in TiMBL, usually demonstrating their functioning using NLP tasks. The IB1-IG algorithm was first introduced in (Daelemans and Van den Bosch, 1992) in the context of a comparison of memory-based approaches with error-backpropagation learning for a hyphenation task. Predecessor versions of IGTREE can be found in (Daelemans and Van den Bosch, 1993; Van den Bosch and Daelemans, 1993) where they are applied to grapheme-to-phoneme conversion. See (Daelemans, Van den Bosch, and Weijters, 1997) for a description and review of IGTREE and IB1-IG. TRIBL is described in (Daelemans, Weijters, and Van den Bosch, 1997b).

Experiments with distance-weighted class voting are described in (Zavrel, 1997). Aspects of using binary-valued (unpacked multi-valued) features are discussed in (Van den Bosch and Zavrel, 2000). Raaijmakers (2000) describes an extension of TiMBL with error-correcting output codes. Hendrickx and Van den Bosch (2004) report on an experiment to import maximum-entropy matrices to replace MVDM matrices (Hendrickx and Van den Bosch, 2004), improving over the maximum-entropy classifier.

Comparisons between memory-based learning and editing variants are reported in (Van den Bosch, 1999; Daelemans, Van den Bosch, and Zavrel, 1999). A hybrid of TiMBL and the RIPPER rule-induction algorithm (Cohen, 1995) is described in (Van den Bosch, 2000; Van den Bosch, 2004a). Using TiMBL as a classifier combination method is discussed in (Van Halteren, Zavrel, and Daelemans, 2001).

Van den Bosch (2004a) presents a search algorithm to find optimal combinations of parameter settings automatically, given a labeled training set of examples, showing large gains of the default settings (also of other machine learning algorithms). Parallelization of TiMBL, through splitting either the training set or the test set in n pieces in shared-memory multi-processor architectures, is explored in (Van den Bosch and Van der Sloot, 2007).

Applications in morpho-phonology

The memory-based algorithms implemented in the TiMBL package have been targeted to a large range of Natural Language Processing tasks. Examples of applications in the **morpho-phonological** area are hyphenation and syllabification (Daelemans and Van den Bosch, 1992); classifying phonemes in speech (Kocsor et al., 2000); assignment of word stress (Daelemans, Gillis, and Durieux, 1994); grapheme-to-phoneme conversion, (Van den Bosch and Daelemans, 1993; Daelemans and Van den Bosch, 1996; Canisius, Van den Bosch, and Daelemans, 2006); diminutive formation (Daelemans et al., 1998); and morphological analysis (Van den Bosch, Daelemans, and Weijters, 1996; Van den Bosch and Daelemans, 1999; Canisius, Van den Bosch, and Daelemans, 2006). Although these examples are applied mostly to Germanic languages (English, Dutch, and German), applications to other languages with more complicated writing systems or morphologies, or with limited resources, have also been presented: for example, letter-phoneme conversion in Scottish

Gaelic (Wolters and Van den Bosch, 1997), morphological analysis of Arabic (Marsi, Van den Bosch, and Soudi, 2006), or diacritic restoration in languages with a diacritic-rich writing system (Mihalcea, 2002; De Pauw, Waiganjo, and De Schryver, 2007).

Linguistic and psycholinguistic modelling

Whereas most work using TiMBL has been oriented towards natural language engineering applications, the **linguistic** and **psycholinguistic** relevance of memory-based learning is another focus of research in Antwerp, Tilburg and elsewhere. Work in this area has been done on stress assignment in Dutch simplex words (Daelemans, Gillis, and Durieux, 1994; Gillis, Durieux, and Daelemans, 2000) and English compounds (Plag, Kunter, and Lappe, 2007), reading aloud (Van den Bosch and Daelemans, 2000), phonological bootstrapping (Durieux and Gillis, 2000), the prediction of linking morphemes in Dutch (Krott, Baayen, and Schreuder, 2001), morphology (Eddington, 2000; Eddington, 2003), and the Dutch plural inflection (Keuleers et al., 2007), including more recent work modelling the production of variable plural forms in Dutch (Zee et al., 2021). Van Craenenbroeck, Van Koppen, and Van den Bosch (2019) use k -nearest neighbor classification in TiMBL to analyse syntactic microvariation in the word order of Dutch verb clusters, and Van Craenenbroeck and Van Koppen (2023) apply leave-one-out k -nearest neighbor classification in TiMBL to study morphosyntactic variation across some 260 Dutch dialects. Vandekerckhove, Sandra, and Daelemans (2013) use TiMBL to develop a language bigram model and use this to explain behavior of patients with a specific impairment that prohibits them to assess adjective orderings as normal or less felicitous.

A comparison to other analogical methods for linguistics is provided in (Daelemans, Weijters, and Van den Bosch, 1997a; Daelemans, 2002). Van den Bosch and Daelemans (2013) offer links to work in psychology on models of human memory, in particular to episodic memory and fast memory access with global matching models (Clark and Gronlund, 1996).

Applications in syntax and semantics

At the **syntactic** sentence level TiMBL has been applied to Part-of-Speech tagging (Daelemans et al., 1996; Zavrel and Daelemans, 1999; Van Halteren, Zavrel, and Daelemans, 2001; Griebenouw, Drevin, and Snyman, 2019); PP-attachment (Zavrel, Daelemans, and Veenstra, 1997); subcategorization (Buchholz, 1998); phrase chunking (Veenstra, 1998; Tjong Kim Sang and Veenstra, 1999); multi-level sequence labeling (Chen and Chen, 2019); shallow parsing of English (Daelemans, Buchholz, and Veenstra, 1999; Buchholz, Veenstra, and Daelemans, 1999; Yeh, 2000) and Arabic (Azmi and bin Badia, 2010); clause identification (Orăsan, 2000; Tjong Kim Sang, 2001); detecting the scope of negation markers and hedge cues (Morante, Liekens, and Daelemans, 2008; Morante and Daelemans, 2009); sentence-boundary detection (Stevenson and Gaizauskas, 2000); and, beyond the sentence level, to co-reference resolution (Preiss, 2002; Mitkov, Evans, and Orasan, 2002; Hoste, 2005; Klenner and Ailloud, 2008; Wunsch, Kübler, and Cantrell, 2009; Zhekova and Kübler, 2011).

Memory-based learning has been integrated as a classifier engine in more complex dependency parsing systems (Nivre, Hall, and Nilsson, 2004; Sagae and Lavie, 2005; Canisius et al., 2006), and dependency parsing in combination with semantic role labeling (Morante, Van Asch, and Van den Bosch, 2009).

Other applications

TiMBL has been involved in several subtasks of **natural language generation**: predicting the order of prenominal adjectives for generation (Malouf, 2000), article generation (Minnen, Bond, and Copestake, 2000); preposition generation (Lee and Knutsson, 2008); generating referring expressions (Hendrickx et al., 2008); and ranking paraphrases of noun compounds (Wubben, 2010).

Memory-based learning has been applied successfully to **lexical semantics**, in particular to word sense disambiguation (Veenstra et al., 2000; Stevenson and Wilks, 1999; Kokkinakis, 2000; Mihalcea, 2002; Hoste et al., 2002; Decadt et al., 2004), but also in other lexical semantic tasks such as determining noun countability (Baldwin and Bond, 2003), animacy (Orăsan and Evans, 2001), semantic relations within noun compounds (Kim and Baldwin, 2006; Nastase et al., 2006), and cross-linguistic word sense disambiguation, a subtask of machine translation (Van Gompel, 2010; Van Gompel and Van den Bosch, 2013; Van Gompel and Van den Bosch, 2014).

As a tool for **text mining**, TiMBL has been used for named-entity recognition (Buchholz and Van den Bosch, 2000; Hendrickx and Van den Bosch, 2003; De Meulder and Daelemans, 2003; Sporleder et al., 2006a; Leveling and Hartrumpf, 2007; Van den Bosch and Bogers, 2013), information extraction (Zavrel, Berck, and Lavrijssen, 2000; Zavrel and Daelemans, 2003; Ahn, 2006), event extraction (Morante, Van Asch, and Daelemans, 2009), text classification (Spitters, 2000), stance detection (Shenoy, Dsouza, and Kübler, 2017), the automatic classification of the mood of songs from their lyrics (Van Zaanen and Kanter, 2010), question classification (Cumbreras, López, and Santiago, 2006; Dridan and Baldwin, 2007), spam filtering (Androutsopoulos et al., 2000), and authorship attribution (Luyckx and Daelemans, 2008; Kestemont and Van Dalen-Oskam, 2009).

In the field of **discourse and dialogue**, TiMBL has been used for dialogue act classification and shallow semantic analysis of speech-recognised utterances (Gustafson, Lindberg, and Lundberg, 1999; Krahmer et al., 2001; Van den Bosch, Krahmer, and Swerts, 2001; Lendvai et al., 2002; Lendvai, Van den Bosch, and Krahmer, 2003a), in disfluency detection in transcribed spontaneous speech (Lendvai, Van den Bosch, and Krahmer, 2003b), in classifying ellipsis in dialogue (Fernández, Ginzburg, and Lappin, 2004), and in classifying errors in answers to reading comprehension tasks (Bailey and Meurers, 2008).

Relations to **statistical language modeling**, in particular the interesting equivalence relations with back-off smoothing in probabilistic classifiers, are discussed in (Zavrel and Daelemans, 1997). Relations between classification-based word prediction and statistical language modeling are identified in (Van den Bosch, 2005; Van den Bosch, 2006; Stehouwer and Van Zaanen, 2009). Next-word prediction with TiMBL has been specifically applied to **word completion** (Van den Bosch and Bogers, 2008; Van den Bosch, 2011; Verberne et al., 2012; Stoop and Van den Bosch, 2014). More recently, this memory-based approach to next-token prediction has been scaled up with TiMBL into a transparent, CPU-only, and eco-friendly alternative to neural **large language models**, with next-token prediction accuracy that scales log-linearly with the amount of training data (Van den Bosch et al., 2025).

TiMBL has been shown to be a useful **error detector** and corrector, such as in the context of error detection in textual databases (Sporleder et al., 2006b), in detecting dependency parsing annotation errors (Dickinson, 2009), in confusable disambiguation (Stehouwer and Van den Bosch, 2009), and in preposition and determiner errors (Van den Bosch and Berck, 2012; Van den Bosch and Berck, 2013).

In **machine translation**, k -nearest neighbor classification offers a conceptual bridge between example-based machine translation (EBMT) and statistical MT. Pure memory-based approaches are described in (Van den Bosch, Stroppa, and Way, 2007; Canisius and Van den Bosch, 2009; Van den Bosch and Berck, 2009; Van Gompel, Van den Bosch, and Berck, 2009); hybrids with statistical

machine translation are presented in (Stroppa, Van den Bosch, and Way, 2007; Haque et al., 2009; Haque et al., 2010); sub-sentential paraphrasing using pivot translations is described in (Max, 2009). In related translation tasks, Tezcan, Hoste, and Macken (2016) include TiMBL among the learners in their machine-translation quality-estimation system for the WMT16 shared task, and Daems, De Clercq, and Macken (2017) train a TiMBL classifier to distinguish human translations from post-edited machine translations (“translationese” versus “post-editese”).

Although most of the applications of TiMBL are in the natural language processing and computational linguistics areas, it is occasionally used in other domains as well. Examples are the classification of newborns’ cries (Feier et al., 2014), scaffolded learning by robots (Saunders, Nehaniv, and Dautenhahn, 2006), the detection of clinically important micro-organisms through an electronic nose (Moens et al., 2006), the identification of cattle breeds from microsatellite DNA markers (Jaiswal et al., 2016), and, through the TiMBL-based memory-based shallow parser MBSP, the generation of visual art (De Smedt, Lechat, and Daelemans, 2011).

Dissertations and special issue

The first dissertation-length study devoted to the approach was (Van den Bosch, 1997), in which the approach is compared to alternative learning methods for NLP tasks related to English word pronunciation (stress assignment, syllabification, morphological analysis, alignment, grapheme-to-phoneme conversion). TiMBL is also central in the Ph.D. theses of Buchholz (2002), Lendvai (2004), Hendrickx (2005), Hoste (2005), Keuleers (2008), Canisius (2009), Vandekerckhove (2013), and Pijpops (2019).

In 1999 a special issue of the *Journal for Experimental and Theoretical Artificial Intelligence* (Vol. 11(3), edited by Walter Daelemans) was devoted to Memory-Based Language Processing. The introduction to this special issue discusses the inspiration sources and alternative developments related to the memory-based approach taken in TiMBL (Daelemans, 1999).

Many of the papers referred to in this section are available in electronic form from the CLIPS home page: <https://www.uantwerpen.be/en/research-groups/clips/>.

Chapter 6

Software usage and options

6.1 Command line options

The user interacts with TiMBL through the use of command line arguments. When you have installed TiMBL successfully, and you type `timbl` at the command line without any further arguments, it will print an overview of the most basic command line options.

```
TiMBL 7.0 (c) CLS/ILK/CLiPS 1998 - 2026.  
Tilburg Memory Based Learner  
Centre for Language Studies, Radboud University Nijmegen  
CLiPS Computational Linguistics Group, University of Antwerp  
Tue Jun 23 12:00:00 2026  
  
usage: timbl -f data-file {-t test-file}  
or see: timbl -h  
        for all possible options
```

If you are satisfied with all of the default settings, you can proceed with just these basics:

- `-f <datafile>` : supplies the name of the file with the training items.
- `-t <testfile>` : supplies the name of the file with the test items.
- `-h` : prints a glossary of all available command line options.

The presence of a training file will make TiMBL pass through the first two phases of its cycle. In the first phase it examines the contents of the training file, and computes a number of statistics on it (feature weights etc.). In the second phase the instances from the training file are stored in memory. If no test file is specified, the program exits, possibly writing some of the results of learning to files (see below). If there is a test file, the selected classifier, trained on the present training data, is applied to it, and the results are written to a file the name of which is a combination of the name of the test file and a code representing the chosen algorithm settings. TiMBL then reports the percentage of correctly classified test items. The default settings for the classification phase are: a Memory-Based Learner, with Gain Ratio feature weighting, with $k = 1$, and with optimizations for speedy search. If you need to change the settings, because you want to use a different type of classifier, or because you need to make a trade-off between speed and memory-use, then you can use the options that are shown using `-h`. The sections below provide

a reference to the use of these command line arguments, and they are roughly ordered by the type of action that the option has effect on. Note that some options (listed with “+/-”) can be turned on (+) or off (-).

6.1.1 Algorithm and metric selection

-a <n> or <string> : determines the classification algorithm. Possible values are:

- 0 or IB1 – the IB1 (k -NN) algorithm (default). See Sections 5.1 and 5.2.
- 1 or IGTREE – IGTREE, decision-tree-based optimization. See Section 5.3.
- 2 or TRIBL – TRIBL, a hybrid of IB1 and IGTREE. See Section 5.4.
- 3 or IB2 – IB2, incremental edited memory-based learning. See Section 5.5.
- 4 or TRIBL2 – TRIBL2, a non-parameteric version of TRIBL. See Section 5.4.

-m <string> : determines which distance metrics are used for each feature. The format of this string is as follows:

GlobalMetric:MetricRange:MetricRange

Where GlobalMetric is used for all features except for the ones that are assigned other metrics by following the restrictions given by :MetricRange. A range can be written using comma’s for lists, and hyphens for intervals. The metric code can be one of the following nine:

- O – Overlap (default; see Subsection 5.1.1)
- M – Modified value difference (MVDM; see Subsection 5.1.4)
- J – Jeffrey divergence (see Subsection 5.1.4)
- S – Jensen-Shannon divergence (see Subsection 5.1.4)
- D – Dot product (see Subsection 5.1.5)
- C – Cosine distance (see Subsection 5.1.5)
- N – Numeric (for numeric features; see Subsection 5.1.1)
- E – Euclidean distance (for numeric features; see Subsection 5.1.1)
- L – Levenshtein (see Subsection 5.1.1)
- DC – Dice coefficient (see Subsection 5.1.1)
- I – Ignore (ignore specified features)

For example, -mO:N3:I2, 5-7 sets the global metric to overlap, declares the third feature to be numeric, and ignores features 2 and 5, 6, and 7.

Ignore *can* be the global metric; it must be followed by a MetricRange string with metric O, M, J, D, or N specifying in the range which features are *not* ignored.

-w <n> : chooses between feature-weighting possibilities. The weights are used in the metric of IB1 and in the ordering of the IGTREE. Possible values are:

- w0 or -w nw – No weighting, i.e. all features have the same importance (weight = 1).
- w1 or -w gr – Gain Ratio weighting (default). See section 5.1.2.
- w2 or -w ig – Information Gain weighting. See section 5.1.2.
- w3 or -w x2 – Chi-squared (χ^2) weighting. See section 5.1.3.
- w4 or -w sv – Shared Variance weighting. See section 5.1.3.
- w5 or -w sd – Standard Deviation weighting.

- `n=<filename>:<number>` or `n=<filename>` – Instead of the five weight settings above we can supply a filename to the `-w` option. This causes TiMBL to read this file and use its contents as weights. If only `<filename>` is given as an argument, the file is supposed to contain one list of feature weights for all features. The `<filename>:<number>` option assumes that a weights file generated by TiMBL with the `-w` option (and possibly edited by the user) is read back in; the number refers to one of the five numbers above. See section 6.2.2 for a description of the format of weights files.
- `-k <n>` : number of nearest neighbors used for extrapolation. Only applicable in conjunction with IB1 (`-a 0`), TRIBL (`-a 2`), TRIBL2 (`-a 4`) and IB2 (`-a 3`). The default is 1. Especially with the MVDm metric it is often useful to determine a good value larger than 1 for this parameter (usually an odd number, to avoid ties). Note that due to ties (instances with exactly the same similarity to the test instance) the number of instances used to extrapolate might in fact be much larger than this parameter.
- `-d <val>` : The type of class voting weights that are used for extrapolation from the nearest neighbor set. `val` can be one of:
- `Z` : normal majority voting; all neighbors have equal weight (default).
 - `ID` : Inverse Distance weighting. See Section 5.1.6, Equation 5.17.
 - `IL` : Inverse Linear weighting. See Section 5.1.6, Equation 5.16.
 - `ED : <a> : <b>` : Exponential Decay weighting with decay parameters a (α) and b (β). No spaces are allowed in the string. Parameter b can be left unspecified: `ED : <a>` assumes $\beta = 1$. The syntax used in previous TiMBL versions (`ED<a>`) is still supported but deprecated. See Section 5.1.6, Equation 5.18.
- `-L <n>` : frequency threshold for switching from the MVDm or Jeffrey Divergence to the Overlap distance metric. The default is 1 (never switch). When in a pair of matched values one or both values occur less frequently than n times in the learning material, TiMBL switches from MVDm or Jeffrey Divergence to Overlap. Higher values of n force TiMBL to use the Overlap metric more.
- Only applicable in conjunction with the MVDm (`-mM`) and Jeffrey divergence (`-mJ`) or Jensen-Shannon divergence (`-mS`) distance metrics.
- `-b <n>` : determines n (≥ 1), the number of instances, to be taken from the top of the training file, to act as the bootstrap set of memorized instances before IB2 starts adding new instances. Only applicable in conjunction with IB2 (`-a 3`).
- `-q <n>` : n is the TRIBL offset, the index number of the feature (counting from 1) after which TRIBL should switch from IGTREE to IB1. Only applicable in conjunction with TRIBL (`-a 2`).
- `-R <n>` : Resolve ties in the classifier randomly, using a random generator with seed n . `-R <n>` causes the classification to be based on a random pick (with seed n) of a category according to the probability distribution in the nearest neighbor set. By default, `-R` is not used, but rather the deterministic tie resolution scheme described in Subsection 5.1.1.
- `-t <@file>` : If the filename given after `-t` starts with '@', TiMBL will read commands for testing from `file`. This file should contain one set of instructions per line. On each line new values can be set for the following command line options: `-B -d -e -k -L -M -o -p -Q -R -t -u +/-v -w +/-x +/-%`. It is compulsory that each line in `file` contains a `-t <testfile>` argument to specify the name of the test file.
- `-t <testfile>` : the string `<testfile>` is the literal name of the file with the test items.

- t `leave_one_out` : No test file is read, but testing is done on each pattern of the training file, by treating each pattern of the training file in turn as a test case (and the whole remainder of the file as training cases). Only applicable in conjunction with IB1 (`-a0`).
- t `cross.validate` : An n -fold cross-validation experiment is performed on the basis of n files (e.g. $1/n$ partitionings of an original data file). The names of these n files need to be in a text file (one name per line) which is given as argument of `-f`. In each fold $f = 1 \dots n$, file number f is taken as test set, and the remaining $n - 1$ files are concatenated to form the training set. Only applicable in conjunction with IB1 (`-a0`).
- T `<n>` : The n th column in the training set of labeled examples contains the label to be predicted, while all other columns represent the input features. By default, the last column is assumed to contain the class labels.

6.1.2 Input options

- f `<datafile>` : the string `<datafile>` is the literal name of the file with the training items, or (in conjunction with `-t cross.validate`, the file containing the names of the cross-validation files.
- F `<format>` : Force TiMBL to interpret the training and test file as a specific data format. Possible values for this parameter are: `Compact`, `C4.5`, `ARFF`, `Columns`, `Tabbed`, `Sparse`, `Binary` (case-insensitive). The default is that TiMBL guesses the format from the contents of the first line of the data file. `ARFF` is not automatically detected. In the `Tabbed` format, fields are separated by single tab characters, so that spaces inside a feature-value or category symbol are significant. See section 6.2.1 for description of the data formats and the guessing rules. The `Compact` format cannot be used with numeric features.
- limit `<l>` : Restrict classification to the `<l>` features that have the highest weights. TiMBL automatically restarts with an adapted `-m` option that ignores the remaining features. This option cannot be combined with cross-validation.
- l `<n>` : Feature length. Only applicable with the `Compact` data format; `<n>` is the number of characters used for each feature-value and category symbol.
- i `<treefile>` : Skip the first two training phases: instead of processing a training file, read a previously saved (see `-I` option) instance-base or IGTREE from the file `treefile`. See section 6.2.4 for the format of this file. Both `-i` and `-f` may be used without `-t`, for instance to build and save an instance base without classifying a test set in the same run.
- matrixin=`<filename>` : Read value distance metrics (such as MVDM or Jeffrey divergence matrices written to file with `--matrixout=<filename>`, or from user-generated matrices) from file `filename`.
- u `<valueclassprobfile>` : Replace the automatically computed value-class probability matrix with the matrices provided in this file.
- P `<path>` : Specify a path to read the data files from. This path is ignored if the name of the data file already contains path information.
- s : Use the whitespace-delimited exemplar weights, given after each training instance in the training file `<datafile>`, during classification. `<testfile>` may contain exemplar weights, but they are not used in classification. If the test file does not have an exemplar weights column, you must specify `-s1`. Exemplar weights can also be ignored (in both training and test files) by specifying `-s0`. Does not work simultaneously with `--occurrences`.

`--occurrences={train|test|both}` : Use the whitespace-delimited integers, given after each instances in a training or test file (or both when training and test actions are invoked at once), as representing the number of occurrences that instance should be counted as. By default these counts are not given, in which case TiMBL counts every instance as a single token. Does not work simultaneously with `-s`.

6.1.3 Output options

`-I <treefile>` : After phase two of learning, save the resulting tree-based representation of the instance-base or IGTREE in a file. This file can later be read back in using the `-i` option (see above). For IGTREE this also automatically saves the current weights into `treefile.wgt` unless this is overridden by `-W`. See section 6.2.4 for a description of the resulting file's format.

`--prune` : Prune an IGTREE instance base directly from an IB1/TRIBL2 instance base, instead of after a full training run. TiMBL reads an existing instance base (`-i`), optionally adds new training material (`-f`), prunes the resulting tree, and writes it to the file given with `-I` (which is therefore required).

`--matrixout=<filename>` : Store calculated MVDM or Jeffrey divergence distance metrics in file `filename`.

`-X <xmlfile>` : instead of the proprietary file format written with the `-I` switch, `-X` writes the TiMBL tree into an XML tree in `<xmlfile>`. This XML file cannot be read back into TiMBL.

`-W <file>` : Save the currently used feature-weights in a file.

`-U <valueclassprobfile>` : Write the automatically computed value-class probability matrix to this file.

`-n <file>` : Save the feature-value and target category symbols in a C4.5 style "names file" with the name `<file>`. Take caution of the fact that TiMBL does not mind creating a file with `'`, `'`, `'` and `'` values in features; C4.5 will produce errors on this.

`-p <n>` : Indicate progress during training and testing after every `n` processed patterns. The default setting is 100,000.

`-e <n>` : During testing, compute and print an estimate on how long it will take to classify `n` test patterns. Off by default.

`+/-v <n>` : Verbosity Level; determines how much information is written to the output during a run. Unless indicated otherwise, this information is written to standard error. The use of `+` turns a given verbosity level **on**, whereas `-` turns it **off** (only useable in non-commandline contexts, such as client/server communication or `-t @testcommandfile`). This parameter can take on the following values (case-insensitive):

`s` : work silently (turns off all set verbosity levels).

`o` : show all options set.

`b` : show node/branch count and branching factor.

`f` : show calculated feature weights. (default)

`p` : show MVDM matrices.

`e` : show exact matches.

`as` : show overall advanced statistics (micro and macro averages of F-score and AUC).

- `cm` : show confusion matrix between actual and predicted classes.
- `cs` : show per-class statistics (precision, recall, true positive rate, false positive rate, F-score, AUC).
- `di` : add the distance of the nearest neighbor to the output file.
- `db` : add class distribution in the nearest neighbor set to the output file.
- `md` : add matching depth and node type (N for non-ending node, L for leaf) to output file.
- `k` : add a summary of class distribution information of all nearest neighbors to the output file (sets `-x`)
- `n` : add nearest neighbors to the output file (sets `-x`)

You may combine levels using '+' e.g. `+v p+db` or `-v o+di`.

- `-G <n>` : Normalize class distributions generated by `+v db`.
 - `0 (zero)` : Normalize distributions so that they add up to 1.0
 - `1:<f>` : Smooth by adding floating-point f to all class votes (e.g. `-G1:1` performs add-one smoothing).
- `--Beam=<n>` : Limit the number of returned classes and class votes returned by `+v db` to n . Default is infinity (no limit).
- `+/- %` : Write the percentage of correctly classified test instances, the number of correctly classified instances, and the total number of classified instances (one number per line, three lines in total) to a file with the same name as the output file, but with the suffix `".%"`.
- `-o <filename>` : Write the test output to filename. Useful for different runs with the same settings on the same testfile, where the default output file name would normally be the same.
- `-O <path>` : Write all output to the path given here. The default is to write all output to the directory where the test file is located.
- `-V` : Show the TiMBL version number.

6.1.4 Internal representation options

- `-N <n>` : (maximum) number of features. Obligatory for Sparse and Binary formats. When larger than a pre-defined constant (default 2500), N needs to be set explicitly for all algorithms.
- `+/-x` : turns the shortcut search for exact matches on or off in IB1 (and IB2, TRIBL, and TRIBL2). The default is off (`-x`). Turning it on makes IB1 generally faster, but with $k > 1$ the shortcut produces different results from a genuine k nearest neighbors search, since absolute preference is given to the exact match.
- `-M <n>` : Set the maximum number of nearest neighbors printed using the `+vn` verbosity option. By default this is set to 500, but when you are interested in the contents of really large nearest neighbor sets (which is possible with large k or large data sets with few features), n can be increased up to 100,000.
- `+/-H` : Turn on/off hashing of feature values and class labels in TiMBL trees. Hashing is done by default, but with short (e.g. one-character) feature values and/or classes less memory is used when hashing is set off.

- +/-D : Store class distributions on all nodes in the TiMBL tree. Default is off (-D). Setting +D costs memory, and is only necessary when using IGTREE in combination with the +v db verbosity setting (which prints class distributions in the classifier output).
- B <n> : Number of bins used for discretizing numeric data (only used for computing feature weights).
- c <n> : Clipping (threshold) frequency for prestoring MVDM matrices. Cells in the matrix are only stored if both feature values occur more than <n> times.
- clones=<n> : Classify a test set using n processors in parallel.
- Treeorder=<string> : Set the ordering of the TiMBL tree (with IB1 and IB2), i.e., rank the features according to the metric identified by <string>. The default ordering is G/V (according to gain ratio divided by the number of values), but some orderings may produce faster classification. Note that different orderings do *not* change the classification behavior of IB1 and IB2. <string> can take the following values:
 - DO : no ordering (the ordering of the features in the data file is taken)
 - GRO : gain ratio (eq. 5.5)
 - IGO : information gain (eq. 5.4)
 - 1/V : $1/V$, where V is the number of values
 - G/V : gain ratio divided by the number of values
 - I/V : information gain divided by the number of values
 - X2O : χ^2 (eq. 5.8)
 - X/V : χ^2 divided by the number of values
 - SVO : shared variance (eq. 5.10)
 - S/V : shared variance divided by the number of values
 - GxE : gain ratio $\times si$, where si is the split info of the feature (eq. 5.6)
 - IxE : information gain $\times si$
 - 1/S : $1/si$

6.1.5 Hidden options

The commandline interface to TiMBL contains several hidden options that have been built in over time for particular reasons. Some have survived over time, and although their use is not for the faint-hearted, some may offer interesting functionalities. A small list of disclosed hidden options follows.

- sloppy={true|false}: in combination with leave-one-out (LOO) testing, this option turns off all weight recomputation. By default, leaving out one training example out causes all feature weights, value-class matrices, and derived metrics such as MVDM to be recomputed, because strictly the example-specific statistics should be absent when it is held out and classified. --sloppy skips this, causing a significant speedup, and usually slightly better LOO scores. Use only if your experimental method allows it. Default value is false.
- silly={true|false}: set to true, switches off the optimized nearest-neighbor search in IB1 and TRIBL. This causes TiMBL to compare all feature values of a test instance to full paths in the TiMBL tree. This causes TiMBL to slow down dramatically on most datasets. Setting is available to enable testing the effect of optimized search. Default value is false.

--Diversify: modifies all features weights by subtracting the smallest weight (plus ϵ) from all weights. The smallest weight thus becomes ϵ . This modification “diversifies” the feature weights, and was introduced to enhance the effect of DIMBL, the multi-CPU variant of TiMBL¹.

6.2 File formats

This section describes the format of the input and output files used by TiMBL. Where possible, the format is illustrated using the classical “objects” data set, which consists of 12 instances of 5 different everyday objects (nut, screw, key, pen, scissors), described by 3 discrete features (size, shape, and number of holes).

6.2.1 Data files

The training and test sets for the learner consist of descriptions of instances in terms of a fixed number of feature-values. TiMBL supports a number of different formats for the instances, but they all have in common that the files should contain one instance per line. The number of instances is determined automatically, and the format of each instance is inferred from the format of the first line in the training set. The last feature of the instance is assumed to be the target category². Should the guess of the format by TiMBL turn out to be wrong, you can force it to interpret the data as a particular format by using the `-F` option. Note that TiMBL, by default, will interpret features as having *symbolic, discrete values*. Unless you specify explicitly that certain features are numeric, using the `-m` option, TiMBL will interpret numbers as just another string of characters. If a feature is numeric, its values will be scaled to the interval $[0,1]$ for purposes of distance computation (see Equation 5.2). The computation of feature weights will be based on a discretization of the feature.

Once TiMBL has determined the input format, it will skip and complain about all lines in the input which do not respect this format (e.g. have a different number of feature-values with respect to that format).

During testing, TiMBL writes the classifications of the test set to an output file. The format of this output file is by default the same as the input format, with the addition of the predicted category being appended after the correct category. If we turn on higher levels of verbosity, the output files will also contain distributions, distances and nearest neighbor sets.

Column format

The **column format** uses white space as the separator between features. White space is defined as a sequence of one or more spaces or tab characters. Every instance of white space is interpreted as a feature separator, so it is not possible to have feature-values containing white space. The column format is auto-detected when an instance of white space is detected on the first line before a comma has been encountered. The example data set looks like this in the column format:

```
small compact 1 nut
small long none screw
```

¹For DIMBL, see <https://github.com/LanguageMachines/dimbl>

²unless the `-s` or `--occurrences` options are used, which both assume a final column with numbers; the class is then the before-last feature.

```

small long 1 key
small compact 1 nut
large long 1 key
small compact none screw
small compact 1 nut
large long none pen
large long 2 scissors
large long 1 pen
large other 2 scissors
small other 2 key

```

C4.5 format

This format is a derivative of the format that is used by the well-known C4.5 decision tree learning program (Quinlan, 1993). The separator between the features is a comma, and the category (viz. the last feature on the line) is followed by a period (although this is not mandatory: TiMBL is robust to missing periods)³. White space within the line is taken literally, so the pattern `a, b c, d` will be interpreted as `'a', ' b c', 'd'`. An exception is the class label, which should not contain any whitespace. When using this format, especially with linguistic data sets or with data sets containing floating point numbers, one should take special care that commas do not occur as feature values and that periods do not occur within the category. Note that TiMBL's C4.5 format does not require a so called *namesfile*. However, TiMBL can produce such a file for C4.5 with the `-n` option. The C4.5 format is auto-detected when a comma is detected on the first line before any white space has been encountered. The example data set looks like this in the C4.5 format:

```

small,compact,1,nut.
small,long,none,screw.
small,long,1,key.
small,compact,1,nut.
large,long,1,key.
small,compact,none,screw.
small,compact,1,nut.
large,long,none,pen.
large,long,2,scissors.
large,long,1,pen.
large,other,2,scissors.
small,other,2,key.

```

ARFF format

ARFF is a format that is used by the WEKA machine learning workbench (Garner, 1995; Witten and Frank, 1999)⁴. Although TiMBL at present does not entirely follow the ARFF specification, it still tries to do as well as it can in reading this format. The ARFF format is not autodetected, and needs to be specified on the commandline with `-F ARFF`.

In ARFF data, the actual data are preceded by a information on feature types, feature names, and names of values in case of symbolic features. TiMBL ignores all of these lines, and starts reading data from after the `@data` statement until the end of the file. Feature-values are supposed to be separated by commas; white space is deleted entirely, so the pattern `a, b c, d` will be interpreted as `'a', 'bc', 'd'`. There should be no whitespace in class labels.

% There are 4 attributes.

³The periods after the category are not reproduced in the output

⁴WEKA is available from the Waikato University Department of Computer Science, <http://www.cs.waikato.ac.nz/~ml/weka>

```
% There are 12 instances.
% Attribute information:
%           'size'           Ints Reals  Enum  Miss
%           'shape'          0      0    12    0
%           'n_holes'        9      0     3    0
%           'class.'         0      0    12    0
@relation 'example.data'
@attribute 'size' { small, large}
@attribute 'shape' { compact, long, other}
@attribute 'n_holes' { 1, none, 2}
@attribute 'class.' { nut., screw., key., pen., scissors.}
@data
small,compact,1,nut.
small,long,none,screw.
small,long,1,key.
small,compact,1,nut.
large,long,1,key.
small,compact,none,screw.
small,compact,1,nut.
large,long,none,pen.
large,long,2,scissors.
large,long,1,pen.
large,other,2,scissors.
small,other,2,key.
```

Compact format

The compact format is especially useful when dealing with very large data files. Because this format does not use any feature separators, file size is reduced considerably in some cases. The price of this is that all features and class labels must be of equal length (in characters) and TiMBL needs to know beforehand what this length is. You must tell TiMBL by using the `-l` option. The compact format is auto-detected when neither of the other formats applies. The same example data set might look like this in the column format with two characters per feature:

```
smcol_nu
smlonosc
smlo1_ke
smcol_nu
lalo1_ke
smconosc
smcol_nu
lalonope
lalo2_sc
lalo1_pe
laot2_sc
smot2_ke
```

Sparse format

The sparse format is relevant for data with features of which a significant portion of the values is 0.0 (numeric), 0 (binary), or some “null” symbolic value. Storing only the non-null values typically takes less space on disk.

Consider, for example, a data set in text classification with 10,000 features each representing the `tf*idf` weight of a term. It would be uneconomical to store instances as long lines of

```
0.02, 0.0, 0.0, 0.0, 0.54, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... , 0.01,sports
```

The Sparse format allows to store such an instance as

```
(1,0.02) (5,0.54) ... (10000,0.01) sports
```

That is, a sequence of (*< index >*, *< value >*) expressions between parentheses each indicating that the feature number *index* has value *value*, with the class label at the end, directly following the last parenthesis. The feature index is assumed to start at 1. In case of symbolic values, whitespace included in the parentheses are considered significant (i.e., part of the values). A case with only null values can be represented as either `'class or , class`.

This option must be specified by the user (`-F Sparse`); it is not guessed from the data. It must also be accompanied by a user declaration of the number of features (`-N <number>`).

Sparse Binary format

The sparse binary format, a simplified version of the Sparse format, is especially useful when dealing with large numbers of two-valued (binary) features, of which each case only has a very few active ones, such as e.g. in text categorization. Thus instead of representing a case as:

```
1,0,0,0,0,0,0,1,0,0,1,1,0,0,0,0,0,1, small.
```

We can represent it as:

```
1,8,11,12,18, small.
```

This format allows one to specify only the index numbers of the active features (indices start at one), while implicitly assuming that the value for all the remaining features is zero. Because each case has a different number of active features, we must specify in some other way what the actual number of features is. This must be done using the `-N` option. As the format is very similar to numeric features, it must always be user-declared using `-F Binary`. The last feature of a line is always interpreted as being the category string. A case with only zeroes can be represented as either `'class or , class`.

6.2.2 Weight files

The feature weights used for computing similarities and for the internal organization of the memory base can be saved to a file `-W`. These files can be read back into TiMBL with `-w <filename>:<weight number>`, where the weight number refers to the five options in TiMBL. It is also possible to change these files manually before reading them in – and additionally it is also possible to write a file from scratch and read this into TiMBL. This allows the experimenter to handcraft feature weights.

The generic format for the weights file is as follows. The weights file may contain comments on lines that start with a `#` character. The other lines contain the number of the feature followed by its numeric weight. An example of such a file is provided below. The numbering of the weights starts with 1. In this example, the data set has three features.

```
# DB Entropy: 2.29248
# Classes: 5
```

```
# Lines of data: 12
# Fea. Weight
1      0.765709
2      0.614222
3      0.73584
```

Weight files written by TiMBL are of the same format, but write all weights in a concatenation, separated by # lines that carry the abbreviated name of the weight (nw, gr, ig, x2, sv). The following example illustrates this format (which can be edited manually, as long as the same number of lines is kept):

```
# DB Entropy: 1.61789
# Classes: 5
# Lines of data: 2999
# nw
# Fea. Weight
1      1
2      1
3      1
#
# gr
# Fea. Weight
1      0.0428445870345557
2      0.185070180760327
3      0.325371814230901
#
# ig
# Fea. Weight
1      0.213887591411729
2      0.669704582861074
3      1.27807624584789
#
# sv
# Fea. Weight
1      0.0762436694064095
2      0.233998145488354
3      0.596896311429044
#
# x2
# Fea. Weight
1      914.619058199289
2      2807.0417532783
3      7160.36815190281
#
```

6.2.3 Value difference files

Using the MVDM metric, it can sometimes be interesting to inspect the matrix of conditional class probabilities from Equation 5.11. By using the `-U` option, we can write the computed matrix to a file. This way we can see which values are considered to be similar by the metric. For each feature a row vector is given for each value, of the conditional probabilities of all the classes (columns) given that value.

```
targets A,      B,      C,      D,      E.

feature # 1 Matrix:
small  0.429  0.286  0.286  0.000  0.000
large  0.000  0.000  0.200  0.400  0.400
```

```

feature # 2 Matrix:
compact 0.750 0.250 0.000 0.000 0.000
long    0.000 0.167 0.333 0.333 0.167
other   0.000 0.000 0.500 0.000 0.500

feature # 3 Matrix:
1        0.500 0.000 0.333 0.167 0.000
none     0.000 0.667 0.000 0.333 0.000
2        0.000 0.000 0.333 0.000 0.667

```

As long as this format is observed, the file can be modified (manually or by substituting other vector-based representations for the values), and the new matrix can be read in and used with the MVDM metric with the `-u` option.

6.2.4 Tree files

Although the learning phase in TiMBL is relatively fast, it can be useful to store the internal representation of the data set both for later usage and for faster subsequent learning. In TiMBL, the data set is stored internally in a tree structure (see Section 5.2). When using IB1, this tree representation contains all the training cases as full paths in the tree. When using IGTREE, unambiguous paths in the tree are pruned before it is used for classification or written to file; on the same data, IGTREE trees are usually considerably smaller than IB1 trees. In either tree type, the arcs represent feature values and nodes contain class distribution information. The features are in the same order throughout the tree. This order is either determined by memory-size considerations in IB1, or by feature relevance in IGTREE. It can explicitly be manipulated using the `-T` option.

We strongly advise to refrain from manually editing the tree file. However, the syntax of the tree file is as follows. First a header consisting of information about the status of the tree, the feature-ordering (the permutation from the order in the data file to the order in the tree), and the presence of numeric features is provided⁵. Subsequently, unless hashing has been set off explicitly (`-H`), a legenda is given of numeric hash codes for the class names (one unique integer per class) and feature value names (one unique integer per value). Subsequently, the tree's nodes and arcs are given in a proprietary non-indented bracket notation.

Starting from the root node, each node is denoted by an opening parenthesis “(”, followed by an integer coding the default class. After this, there is the class distribution list, within curly braces “{ }”, containing a non-empty list of category codes followed by integer counts. After this comes an optional comma-separated list of arcs to child nodes, within “[]” brackets. An arc is labeled with a coded feature value. The node that the arc leads to again has a class distribution, and any number of child nodes pointed to by arcs.

The IB1 tree constructed from our example data set looks as follows:

```

# Status: complete
# Permutation: < 1, 3, 2 >
# Numeric: .
# Version 4 (Hashed)
#
Classes
1      nut
2      screw
3      key
4      pen
5      scissors
Features

```

⁵Although in this header each line starts with ‘#’, these lines cannot be seen as comment lines.

```

1      small
2      compact
3      1
4      long
5      none
6      large
7      2
8      other

```

```

(1{ 1 3, 2 2, 3 3, 4 2, 5 2 }[1(1[3(1[2(1{ 1 3 })
,4(3{ 3 1 })
]
)
,5(2[2(2{ 2 1 })
,4(2{ 2 1 })
]
)
,7(3[8(3{ 3 1 })
]
)
]
)
,6(4[3(3[4(3{ 3 1, 4 1 })
]
)
,5(4[4(4{ 4 1 })
]
)
,7(5[4(5{ 5 1 })
,8(5{ 5 1 })
]
)
]
)
]
)
)

```

The corresponding compressed IGTREE version is considerably smaller.

```

# Status: pruned
# Permutation: < 1, 3, 2 >
# Numeric: .
# Version 4 (Hashed)
#
Classes
1      nut
2      screw
3      key
4      pen
5      scissors
Features
1      small
2      compact
3      1
4      long
5      none
6      large
7      2
8      other

(1{ 1 3, 2 2, 3 3, 4 2, 5 2 }[1(1{ 1 3, 2 2, 3 2 }[3(1{ 1 3, 3 1 }[4(3{ 3 1 })
]
)
,5(2{ 2 2 })
,7(3{ 3 1 })

```

```
]
)
,6(4{ 3 1, 4 2, 5 2 }[3(3{ 3 1, 4 1 })
,7(5{ 5 2 })
]
)
]
)
```

TiMBL tree files generated by versions 1.0 to 3.0 of TiMBL, which do not contain hashed class and value names, are no longer recognized in current TiMBL versions. Backward compatibility to trees generated by versions 1.0 to 3.0 is preserved in TiMBL version 4 up to release 4.3.1.

References

- Aha, D. W. 1997. Lazy learning: Special issue editorial. *Artificial Intelligence Review*, 11:7–10.
- Aha, D. W., D. Kibler, and M. Albert. 1991. Instance-based learning algorithms. *Machine Learning*, 6:37–66.
- Ahn, D. 2006. The stages of event extraction. In *Proceedings of the COLING-ACL 2006 Workshop on Annotating and Reasoning about Time and Events*, pages 1–8, Sydney, Australia.
- Androutsopoulos, I., G. Paliouras, V. Karkaletsis, G. Sakkis, C. D. Spyropoulos, and P. Stamatoopoulos. 2000. Learning to filter spam e-mail: A comparison of a Naive Bayesian and a memory-based approach. In *Proceedings of the "Machine Learning and Textual Information Access" Workshop of the 4th European Conference on Principles and Practice of Knowledge Discovery in Databases*.
- Azmi, A. M. and N. bin Badia. 2010. e-Narrator – an application for creating an ontology of *hadiths* narration tree semantically and graphically. *The Arabian Journal for Science and Engineering*, 35(2C).
- Baayen, R. H., R. Piepenbrock, and H. van Rijn. 1993. *The CELEX lexical data base on CD-ROM*. Linguistic Data Consortium, Philadelphia, PA.
- Bailey, S. and D. Meurers. 2008. Diagnosing meaning errors in short answers to reading comprehension questions. In *Proceedings of the Third Workshop on Innovative Use of NLP for Building Educational Applications*, pages 107–115, Columbus, Ohio, June. Association for Computational Linguistics.
- Bailey, T. and A. K. Jain. 1978. A note on distance-weighted k -nearest neighbor rules. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-8(4):311–313.
- Baldwin, T. and F. Bond. 2003. A plethora of methods for learning English countability. In M. Collins and M. Steedman, editors, *Proceedings of the 2003 Conference on Empirical Methods in Natural Language Processing*, pages 73–80, New Brunswick, NJ. ACL.
- Buchholz, S. 1998. Distinguishing complements from adjuncts using memory-based learning. In *Proceedings of the ESSLLI-98 Workshop on Automated Acquisition of Syntax and Parsing*, Saarbrücken, Germany.
- Buchholz, S. 2002. *Memory-Based Grammatical Relation Finding*. PhD thesis, University of Tilburg.
- Buchholz, S. and A. Van den Bosch. 2000. Integrating seed names and n-grams for a named entity list and classifier. In *Proceedings of the Second International Conference on Language Resources and Evaluation*, pages 1215–1221, Athens, Greece.
- Buchholz, S., J. Veenstra, and W. Daelemans. 1999. Cascaded grammatical relation assignment. In *EMNLP-VLC'99, the Joint SIGDAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora*, June.
- Canisius, S. 2009. *Structured prediction for natural language processing: A constraint satisfaction approach*. Ph.D. thesis, Tilburg University.
- Canisius, S., T. Bogers, A. Van den Bosch, J. Geertzen, and E. Tjong Kim Sang. 2006. Dependency parsing by inference over high-recall dependency predictions. In *Proceedings of the Tenth Conference on Computational Natural Language Learning, CoNLL-X*, pages 176–180, New York, NY, June.
- Canisius, S. and A. Van den Bosch. 2009. A constraint satisfaction approach to machine translation. In *Proceedings of the 13th Annual Conference of the European Association for Machine Translation (EAMT-2009)*, pages 182–189.

- Canisius, S., A. Van den Bosch, and W. Daelemans. 2006. Constraint satisfaction inference: Non-probabilistic global inference for sequence labelling. In *Proceedings of the EACL 2006 Workshop on Learning Structured Information in Natural Language Applications*, pages 9–16, Trento, Italy.
- Cardie, C. 1996. Automatic feature set selection for case-based learning of linguistic knowledge. In *Proceedings of the Conference on Empirical Methods in NLP*. University of Pennsylvania.
- Chen, Y. and J. Chen. 2019. A k -nearest neighbor approach towards multi-level sequence labeling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Industry Papers)*, pages 149–156, Minneapolis, Minnesota. Association for Computational Linguistics.
- Clark, S. E. and S. D. Gronlund. 1996. Global matching models of recognition memory: How the models match the data. *Psychonomic Bulletin & Review*, 3(1):37–60.
- Cohen, W. 1995. Fast effective rule induction. In *Proceedings of the 12th International Conference on Machine Learning*, pages 115–123. Morgan Kaufmann.
- Cost, S. and S. Salzberg. 1993. A weighted nearest neighbour algorithm for learning with symbolic features. *Machine Learning*, 10:57–78.
- Cover, T. M. and P. E. Hart. 1967. Nearest neighbor pattern classification. *Institute of Electrical and Electronics Engineers Transactions on Information Theory*, 13:21–27.
- Cumbreras, M.A. García, L.A. Ureña López, and F. Martínez Santiago. 2006. BRUJA: Question classification for Spanish. In *Proceedings of the EACL 2006 Workshop on Multilingual Question Answering*, pages 39–44, Trento, Italy.
- Daelemans, W. 1999. Memory-based language processing. *Journal of Experimental and Theoretical Artificial Intelligence*, 11(3):287–296.
- Daelemans, W. 2002. A comparison of analogical modeling to memory-based language processing. In R. Skousen, D. Lonsdale, and D.B. Parkinson, editors, *Analogical Modeling*. John Benjamins, Amsterdam, The Netherlands.
- Daelemans, W., P. Berck, and S. Gillis. 1997. Data mining as a method for linguistic analysis: Dutch diminutives. *Folia Linguistica*, XXXI(1–2):57–75.
- Daelemans, W., S. Buchholz, and J. Veenstra. 1999. Memory-based shallow parsing. In *Proceedings of CoNLL*, Bergen, Norway.
- Daelemans, W., S. Gillis, and G. Durieux. 1994. The acquisition of stress: a data-oriented approach. *Computational Linguistics*, 20(3):421–451.
- Daelemans, W. and A. Van den Bosch. 1992. Generalisation performance of backpropagation learning on a syllabification task. In M. F. J. Drossaers and A. Nijholt, editors, *Proceedings of TWLT3: Connectionism and Natural Language Processing*, pages 27–37, Enschede. Twente University.
- Daelemans, W. and A. Van den Bosch. 1993. TabTalk: reusability in data-oriented grapheme-to-phoneme conversion. In *Proceedings of Eurospeech '93*, pages 1459–1466, Berlin. T.U. Berlin.
- Daelemans, W. and A. Van den Bosch. 1996. Language-independent data-oriented grapheme-to-phoneme conversion. In J. P. H. Van Santen, R. W. Sproat, J. P. Olive, and J. Hirschberg, editors, *Progress in Speech Processing*. Springer-Verlag, Berlin, pages 77–89.
- Daelemans, W. and A. Van den Bosch. 2005. *Memory-based language processing*. Cambridge University Press, Cambridge, UK.

- Daelemans, W., A. Van den Bosch, and A. Weijters. 1997. IGTREE: using trees for compression and classification in lazy learning algorithms. *Artificial Intelligence Review*, 11:407–423.
- Daelemans, W., A. Van den Bosch, and J. Zavrel. 1999. Forgetting exceptions is harmful in language learning. *Machine Learning, Special issue on Natural Language Learning*, 34:11–41.
- Daelemans, W., A. Weijters, and A. Van den Bosch. 1997a. Empirical learning of natural language processing tasks. In M. Van Someren and G. Widmer, editors, *Machine Learning: Proceedings of ECML-97*, number 1224 in Lecture Notes in Artificial Intelligence, pages 337–344, Berlin. Springer-Verlag.
- Daelemans, W., A. Weijters, and A. Van den Bosch, editors. 1997b. *Workshop Notes of the ECML/MLnet familiarisation workshop on Empirical learning of natural language processing tasks*, Prague, Czech Republic. University of Economics.
- Daelemans, W., J. Zavrel, P. Berck, and S. Gillis. 1996. MBT: A memory-based part of speech tagger generator. In E. Ejerhed and I. Dagan, editors, *Proceedings of the Fourth Workshop on Very Large Corpora*, pages 14–27. ACL SIGDAT.
- Daelemans, W., J. Zavrel, K. Van der Sloot, and A. Van den Bosch. 1998. TiMBL: Tilburg memory based learner, version 1.0, reference manual. Technical Report ILK 98-03, ILK Research Group, Tilburg University.
- Daems, J., O. De Clercq, and L. Macken. 2017. Translationese and post-edits: How comparable is comparable quality? *Linguistica Antverpiensia, New Series – Themes in Translation Studies*, 16:89–103.
- De Meulder, F. and W. Daelemans. 2003. Memory-based named entity recognition using unannotated data. In W. Daelemans and M. Osborne, editors, *Proceedings of CoNLL-2003*, pages 208–211. Edmonton, Canada.
- De Pauw, G., P. Waiganjo, and G.-M. De Schryver. 2007. Automatic diacritic restoration for resource-scarce languages. In *Proceedings of Text, Speech and Dialogue, Tenth International Conference*, volume 4629 of *Lecture Notes in Computer Science*, pages 170–179, Berlin, Germany. Springer Verlag.
- De Smedt, T., L. Lechat, and W. Daelemans. 2011. Generative art inspired by nature, using NodeBox. In C. Di Chio et al., editors, *Applications of Evolutionary Computation (EvoApplications 2011)*, Part II, volume 6625 of *Lecture Notes in Computer Science*, pages 264–272, Berlin. Springer.
- Devijver, P. A. and J. Kittler. 1982. *Pattern recognition. A statistical approach*. Prentice-Hall, London, UK.
- Decadt, B., V. Hoste, W. Daelemans, and A. Van den Bosch. 2004. GAMBL, genetic algorithm optimization of memory-based WSD. In R. Mihalcea and P. Edmonds, editors, *Proceedings of the Third International Workshop on the Evaluation of Systems for the Semantic Analysis of Text (Senseval-3)*, pages 108–112, New Brunswick, NJ. ACL.
- Dickinson, M. 2009. Correcting dependency annotation errors. In *Proceedings of the 12th Conference of the European Chapter of the Association for Computational Linguistics*, pages 193–201. Association for Computational Linguistics.
- Dridan, R. and T. Baldwin. 2007. What to classify and how: Experiments in question classification for Japanese. In *Proceedings of the 10th Conference of the Pacific Association for Computational Linguistics*, pages 333–341, Melbourne, Australia.
- Dudani, S.A. 1976. The distance-weighted k -nearest neighbor rule. In *IEEE Transactions on Systems, Man, and Cybernetics*, volume SMC-6, pages 325–327.

- Durieux, G. and S. Gillis. 2000. Predicting grammatical classes from phonological cues: An empirical test. In B. Höhle and J. Weissenborn, editors, *Approaches to bootstrapping: Phonological, syntactic and neurophysiological aspects of early language acquisition*. Benjamins, Amsterdam, The Netherlands, pages 189–232.
- Eddington, David. 2000. Analogy and the dual-route model of morphology. *Lingua*, 110:281–298.
- Eddington, David. 2003. Issues in modeling language processing analogically. *Lingua*, 114:849–871.
- Egan, J. P. 1975. *Signal detection theory and ROC analysis*. Series in Cognition and Perception. Academic Press, New York, NY.
- Fawcett, T. 2004. ROC graphs: Notes and practical considerations for researchers. Technical Report HPL-2003-4, Hewlett Packard Labs.
- Feier, F., I. Enatescu, C. Ilie, and I. Silea. 2014. Newborns’ cry analysis classification using signal processing and data mining. In *Optimization of Electrical and Electronic Equipment (OPTIM), 2014 International Conference on*, pages 880–885. IEEE.
- Fernández, R., J. Ginzburg, and S. Lappin. 2004. Classifying ellipsis in dialogue: A machine learning approach. In *Proceedings of the 20th International Conference on Computational Linguistics, COLING 2004*, pages 240–246, Geneva, Switzerland, August.
- Garner, S.R. 1995. WEKA: The Waikato Environment for Knowledge Analysis. In *Proceedings of the New Zealand Computer Science Research Students Conference*, pages 57–64.
- Gillis, S., G. Durieux, and W. Daelemans. 2000. Lazy learning: A comparison of natural and machine learning of stress. In P. Broeder and J. Murre, editors, *Cognitive Models of Language Acquisition*, pages 76–99. Cambridge University Press.
- Griekenouw, A., G. Drevin, and D. Snyman. 2019. A combination part of speech tagger using selected voting methods. In *2019 International Multidisciplinary Information Technology and Engineering Conference (IMITEC)*, pages 1–6, Vanderbijlpark, South Africa. IEEE.
- Gustafson, J., N. Lindberg, and M. Lundeberg. 1999. The august spoken dialogue system. In *Proceedings of Eurospeech’99*.
- Haque, R., S. Kumar Naskar, A. Van den Bosch, and A. Way. 2009. Dependency relations as source context in phrase-based SMT. In *Proceedings of PACLIC 23: the 23rd Pacific Asia Conference on Language, Information and Computation*, pages 170–179, Hong Kong, China.
- Haque, R., S. Kumar Naskar, A. Van den Bosch, and A. Way. 2010. Supertags as source language context in hierarchical phrase-based SMT. In *AMTA 2010: The Ninth Conference of the Association for Machine Translation in the Americas, Proceedings*, pages 210–219, Denver, CO.
- Hart, P. E. 1968. The condensed nearest neighbor rule. *IEEE Transactions on Information Theory*, 14:515–516.
- Hendrickx, I. 2005. *Local classification and global estimation: Explorations of the k-nearest neighbor algorithm*. Ph.D. thesis, Tilburg University.
- Hendrickx, I., W. Daelemans, K. Luyckx, R. Morante, and V. Van Asch. 2008. CNTS: Memory-based learning of generating repeated references. In *Proceedings of the Fifth International Natural Language Generation Conference*, pages 194–195. Association for Computational Linguistics.
- Hendrickx, I. and A. Van den Bosch. 2003. Memory-based one-step named-entity recognition: Effects of seed list features, classifier stacking, and unannotated data. In *Proceedings of CoNLL-2003*, pages 176–179.

- Hendrickx, I. and A. Van den Bosch. 2004. Maximum-entropy parameter estimation for the k -NN modified value-difference kernel. In R. Verbrugge, N. Taatgen, and L. Schomaker, editors, *Proceedings of the Sixteenth Belgian-Dutch Conference on Artificial Intelligence, Groningen*, pages 19–26, Groningen, The Netherlands.
- Hoste, V. 2005. *Optimization in Machine Learning of Coreference Resolution*. Ph.D. thesis, University of Antwerp.
- Hoste, V., I. Hendrickx, W. Daelemans, and A. Van den Bosch. 2002. Parameter optimization for machine learning of word sense disambiguation. *Natural Language Engineering*, 8(4):311–325.
- Jaiswal, S., S. K. Dhanda, M. A. Iquebal, V. Arora, T. M. Shah, U. B. Angadi, C. G. Joshi, G. P. S. Raghava, A. Rai, and D. Kumar. 2016. BIS-CATTLE: A web server for breed identification using microsatellite DNA markers. *Current Research in Bioinformatics*, 5(1):10–17.
- Kestemont, M. and K. Van Dalen-Oskam. 2009. Predicting the past: memory-based copyist and author discrimination in medieval epics. In *Proceedings of the Twenty-first Benelux Conference on Artificial Intelligence (BNAIC 2009)*, volume 21, pages 121–128.
- Keuleers, E. 2008. *Memory-based learning of inflectional morphology*. Ph.D. thesis, University of Antwerp.
- Keuleers, E., D. Sandra, W. Daelemans, S. Gillis, G. Durieux, and E. Martens. 2007. Dutch plural inflection: The exception that proves the analogy. *Cognitive Psychology*, 54(4):283–318.
- Kim, S.N. and T. Baldwin. 2006. Interpreting semantic relations in noun compounds via verb semantics. In *Proceedings of the COLING/ACL 2006 Main Conference Poster Sessions*, pages 491–498, Sydney, Australia.
- Klenner, M. and E. Ailloud. 2008. Enhancing coreference clustering. In *Proceedings of the Second Workshop on Anaphora Resolution (WAR II)*, volume 2, pages 31–40.
- Kocsor, A., L. Tóth, A. Kuba jr., K. Kovács, M. Jelasity, T. Gyimóthy, and J. Csirik. 2000. A comparative study of several feature transformation and learning methods for phoneme classification. *International Journal of Speech Technology*, 3(3/4):263–276.
- Kokkinakis, D. 2000. PP-attachment disambiguation for Swedish: Combining unsupervised and supervised training data. *Nordic Journal of Linguistics*, 23(2):191–213.
- Kolodner, J. 1993. *Case-based reasoning*. San Mateo, CA: Morgan Kaufmann.
- Krahmer, E., M. Swerts, M. Theune, and M. Weegels. 2001. Error detection in spoken human-machine interaction. *International Journal of Speech Technology*, 4(1):19–30.
- Krott, A., R. H. Baayen, and R. Schreuder. 2001. Analogy in morphology: modeling the choice of linking morphemes in Dutch. *Linguistics*, 39(1):51–93.
- Lee, J. and O. Knutsson. 2008. The role of pp attachment in preposition generation. In A. Gelbukh, editor, *Proceedings of CICLing 2008, 9th International Conference on Intelligent Text Processing and Computational Linguistics*, number 4919 in Lecture Notes in Computer Science, pages 643–654. Springer Verlag.
- Lendvai, P. 2004. *Extracting information from spoken user input: A machine learning approach*. Ph.D. thesis, Tilburg University.
- Lendvai, P., A. Van den Bosch, and E. Krahmer. 2003a. Machine learning for shallow interpretation of user utterances in spoken dialogue systems. In *Proceedings of the EACL Workshop on Dialogue Systems: Interaction, adaptation and styles of management*, pages 69–78.

- Lendvai, P., A. Van den Bosch, and E. Krahmer. 2003b. Memory-based disfluency chunking. In *Proceedings of Disfluency in Spontaneous Speech Workshop (DISS'03)*, pages 63–66.
- Lendvai, P., A. Van den Bosch, E. Krahmer, and M. Swerts. 2002. Improving machine-learned detection of miscommunications in human-machine dialogues through informed data splitting. In *Proceedings of the ESSLLI Workshop on Machine Learning Approaches in Computational Linguistics*.
- Leveling, J. and S. Hartrumpf. 2007. On metonymy recognition for geographic IR. In *Proceedings of GIR-2006, the 3rd Workshop on Geographical Information Retrieval*, Seattle, WA.
- Levenshtein, V. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Sovjet Physics Doklady*, 10:707–710.
- Luyckx, K. and W. Daelemans. 2008. Authorship attribution and verification with many authors and limited data. In *Proceedings of the Twenty-Second International Conference on Computational Linguistics (COLING 2008)*, pages 513–520, Manchester, UK. Coling 2008 Organizing Committee.
- MacLeod, J. E. S., A. Luk, and D. M. Titterton. 1987. A re-examination of the distance-weighted k -nearest neighbor classification rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-17(4):689–696.
- Malouf, R. 2000. The order of prenominal adjectives in natural language generation. In *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics*, pages 85–92, New Brunswick, NJ. ACL.
- Marsi, E., A. Van den Bosch, and A. Soudi. 2006. Memory-based morphological analysis generation and part-of-speech tagging of Arabic. In *Proceedings of the ACL Workshop on Computational Approaches to Semitic Languages*, Ann Arbor, MI.
- Max, A. 2009. Sub-sentential paraphrasing by contextual pivot translation. In *Proceedings of the 2009 Workshop on Applied Textual Inference*, pages 18–26. Association for Computational Linguistics.
- Mihalcea, R. 2002. Instance-based learning with automatic feature selection applied to word sense disambiguation. In *Proceedings of the 19th International Conference on Computational Linguistics (COLING 2002)*, Taipei, Taiwan.
- Minnen, G., F. Bond, and A. Copestake. 2000. Memory-based learning for article generation. In *Proceedings of the 4th Conference on Computational Natural Language Learning and the Second Learning Language in Logic Workshop*, pages 43–48, New Brunswick, NJ. ACL.
- Mitkov, R., R. Evans, and C. Orasan. 2002. A new, fully automatic version of Mitkov's knowledge-poor pronoun resolution method. In *Proceedings of the Third International Conference on Computational Linguistics and Intelligent Text Processing*, pages 168–186. Springer-Verlag.
- Moens, M., A. Smet, B. Naudts, J. Verhoeven, M. Ieven, P. Jorens, H. J. Geise, and F. Blockhuys. 2006. Fast identification of ten clinically important micro-organisms using an electronic nose. *Letters in applied microbiology*, 42(2):121–126.
- Morante, R. and W. Daelemans. 2009. Learning the scope of hedge cues in biomedical texts. In *Proceedings of the Workshop on BioNLP*, pages 28–36. Association for Computational Linguistics.
- Morante, R., A. Liekens, and W. Daelemans. 2008. Learning the scope of negation in biomedical texts. In *Proceedings of the 2008 Conference on Empirical Methods in Natural Language Processing*, pages 715–724.
- Morante, R., V. Van Asch, and W. Daelemans. 2009. A memory-based learning approach to event extraction in biomedical texts. In *Proceedings of the Workshop on BioNLP: Shared Task*, pages 59–67. Association for Computational Linguistics.

- Morante, R., V. Van Asch, and A. Van den Bosch. 2009. Dependency parsing and semantic role labeling as a single task. In *Proceedings of the 7th International Conference on Recent Advances in Natural Language Processing (RANLP-2009)*, pages 275–280, Borovets, Bulgaria.
- Morin, R. L. and B. E. Raeside. 1981. A reappraisal of distance-weighted k -nearest neighbor classification for pattern recognition with missing data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-11(3):241–243.
- Nastase, V., J. Sayyad-Shiarabad, M. Sokolova, and S. Szpakowicz. 2006. Learning noun-modifier semantic relations with corpus-based and wordnet-based features. In *Proceedings of the Twenty-First National Conference on Artificial Intelligence and the Eighteenth Innovative Applications of Artificial Intelligence Conference*. AAAI Press.
- Nivre, J., J. Hall, and J. Nilsson. 2004. Memory-based dependency parsing. In H. T. Ng and E. Riloff, editors, *Proceedings of the Eighth Conference on Computational Natural Language Learning (CoNLL 2004)*, pages 49–57, Boston, MA.
- Orăsan, C. 2000. A hybrid method for clause splitting in unrestricted English texts. In *Proceedings of ACIDCA '2000*, Monastir, Tunisia.
- Orăsan, C. and R. Evans. 2001. Learning to identify animate references. In W. Daelemans and R. Zajac, editors, *Proceedings of the Fifth Workshop on Computational Language Learning, CoNLL-2001*, pages 129–136, Toulouse, France.
- Pijpops, D. 2019. *How, why and where does argument structure vary? A usage-based investigation into the Dutch transitive-prepositional alternation*. Ph.D. thesis, University of Leuven.
- Plag, I., G. Kunter, and S. Lappe. 2007. Testing hypotheses about compound stress assignment in English: a corpus-based investigation. *Corpus Linguistics and Linguistic Theory*, 3(2):199–232.
- Preiss, J. 2002. Anaphora resolution with memory-based learning. In *Proceedings of the Fifth Annual CLUK Research Colloquium*, pages 1–9.
- Quinlan, J.R. 1986. Induction of decision trees. *Machine Learning*, 1:81–206.
- Quinlan, J.R. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA.
- Raaijmakers, S. 2000. Learning distributed linguistic classes. In *Proceedings of the Fourth Conference on Computational Language Learning and the Second Learning Language in Logic Workshop*, pages 55–60, New Brunswick, NJ. ACL.
- Sagae, K. and A. Lavie. 2005. A classifier-based parser with linear run-time complexity. In *Proceedings of the Ninth International Workshop on Parsing Technologies*, pages 125–132, Vancouver, Canada.
- Salzberg, S. 1990. *Learning with nested generalised exemplars*. Kluwer Academic Publishers, Norwell, MA.
- Saunders, J., C. L. Nehaniv, and K. Dautenhahn. 2006. Teaching robots by moulding behavior and scaffolding the environment. In *Proceedings of the 1st ACM SIGCHI/SIGART conference on Human-robot interaction*, pages 118–125. ACM.
- Shenoy, G. G., E. H. Dsouza, and S. Kübler. 2017. Performing stance detection on Twitter data using computational linguistics techniques. *arXiv preprint arXiv:1703.02019*.
- Shepard, R.N. 1987. Toward a universal law of generalization for psychological science. *Science*, 237:1317–1323.
- Spitters, M. 2000. Comparing feature sets for learning text categorization. In *Proceedings of the Sixth Conference on Content-Based Multimedia Access (RIA0 2002)*, pages 1124–1135, Paris, France.

- Sporleder, C., M. Van Erp, T. Porcelijn, and A. Van den Bosch. 2006a. Identifying named entities in text databases from the natural history domain. In *Proceedings of the Fifth International Conference on Language Resources and Evaluation, LREC-2006*, Trento, Italy.
- Sporleder, C., M. Van Erp, T. Porcelijn, and A. Van den Bosch. 2006b. Spotting the ‘odd-one-out’: Data-driven error detection and correction in textual databases. In *Proceedings of the EACL 2006 Workshop on Adaptive Text Extraction and Mining, ATEM-2006*, Trento, Italy.
- Stanfill, C. and D. Waltz. 1986. Toward memory-based reasoning. *Communications of the ACM*, 29(12):1213–1228, December.
- Stehouwer, H. and A. Van den Bosch. 2009. Putting the t where it belongs: Solving a confusion problem in Dutch. In S. Verberne, H. van Halteren, and P.-A. Coppen, editors, *Computational Linguistics in the Netherlands 2007: Selected Papers from the 18th CLIN Meeting*, pages 21–36, Nijmegen, The Netherlands.
- Stehouwer, H. and M. Van Zaanen. 2009. Language models for contextual error detection and correction. In *Proceedings of the EACL 2009 Workshop on Computational Linguistic Aspects of Grammatical Inference*, pages 41–48, Athens, Greece.
- Stevenson, M. and R. Gaizauskas. 2000. Experiments on sentence boundary detection. In *Proceedings of the Sixth Conference on Applied Natural Language Processing and the First Conference of the North American Chapter of the Association for Computational Linguistics*, pages 24–30.
- Stevenson, M. and Y. Wilks. 1999. Combining weak knowledge sources for sense disambiguation. In *Proceedings of the International Joint Conference on Artificial Intelligence*.
- Stoop, W. and A. Van den Bosch. 2014. Using idiolects and sociolects to improve word prediction. In *Proceedings of the 14th Conference of the European Chapter of the Association for Computational Linguistics*, pages 318–327, Gothenburg, Sweden, April. Association for Computational Linguistics.
- Stroppa, N., A. Van den Bosch, and A. Way. 2007. Exploiting source similarity for SMT using context-informed features. In A. Way and B. Gawronska, editors, *Proceedings of the 11th International Conference on Theoretical Issues in Machine Translation (TMI 2007)*, pages 231–240, Skövde, Sweden.
- Swets, J., R. Dawes, and J. Monahan. 2000. Better decisions through science. *Scientific American*, 283:82–87.
- Tezcan, A., V. Hoste, and L. Macken. 2016. UGENT-LT3 SCATE submission for the WMT16 shared task on quality estimation. In *Proceedings of the First Conference on Machine Translation (WMT), Volume 2: Shared Task Papers*, pages 843–850, Berlin, Germany. Association for Computational Linguistics.
- Tjong Kim Sang, E. 2001. Memory-based clause identification. In *Proceedings of CoNLL-2001*, pages 67–69. Toulouse, France.
- Tjong Kim Sang, E. and J. Veenstra. 1999. Representing text chunks. In *Proceedings of EACL’99*, pages 173–179. Bergen, Norway.
- Van Craenenbroeck, J. and M. Van Koppen. 2023. Parameters and language contact: Morphosyntactic variation in Dutch dialects. *Catalan Journal of Linguistics*, 22:71–95.
- Van Craenenbroeck, J., M. Van Koppen, and A. Van den Bosch. 2019. A quantitative-theoretical analysis of syntactic microvariation: Word order in Dutch verb clusters. *Language*, 95(2):333–370.
- Van den Bosch, A. 1997. *Learning to pronounce written words: A study in inductive language learning*. Ph.D. thesis, Universiteit Maastricht, Cadier en Keer, the Netherlands, December.

- Van den Bosch, A. 1999. Careful abstraction from instance families in memory-based language learning. *Journal for Experimental and Theoretical Artificial Intelligence*, 11(3):339–368.
- Van den Bosch, A. 2000. Using induced rules as complex features in memory-based language learning. In *Proceedings of the Fourth Conference on Computational Natural Language Learning and of the Second Learning Language in Logic Workshop*, pages 73–78, New Brunswick, NJ. ACL.
- Van den Bosch, A. 2004a. Feature transformation through rule induction: a case study with the k-nn classifier. In J. Fürnkranz, editor, *Proceedings of the ECML/PKDD 2004 Workshop on Advances in Inductive Rule Learning*, pages 1–16, Pisa, Italy.
- Van den Bosch, A. 2004b. Wrapped progressive sampling search for optimizing learning algorithm parameters. In R. Verbrugge, N. Taatgen, and L. Schomaker, editors, *Proceedings of the Sixteenth Belgian-Dutch Conference on Artificial Intelligence*, pages 219–226, Groningen, The Netherlands.
- Van den Bosch, A. 2005. Memory-based understanding of user utterances in a spoken dialogue system: Effects of feature selection and co-learning. In *Workshop Proceedings of the 6th International Conference on Case-Based Reasoning*, pages 85–94, Chicago, IL.
- Van den Bosch, A. 2006. Spelling space: A computational test bed for phonological and morphological changes in Dutch spelling. *Written Language and Literacy*, 9(1):25–44.
- Van den Bosch, A. 2011. Effects of context and recency in scaled word completion. *Computational Linguistics in the Netherlands Journal*, 1:79–94.
- Van den Bosch, A. and P. Berck. 2009. Memory-based machine translation and language modeling. *The Prague Bulletin of Mathematical Linguistics*, 91:17–26.
- Van den Bosch, A. and P. Berck. 2012. Memory-based text correction for preposition and determiner errors. In *Proceedings of the 7th Workshop on the Innovative Use of NLP for Building Educational Applications*, pages 289–294, New Brunswick, NJ. ACL.
- Van den Bosch, A. and P. Berck. 2013. Memory-based grammatical error correction. In *Proceedings of the Seventeenth Conference on Computational Natural Language Learning: Shared Task*, pages 102–108, Sofia, Bulgaria, August. Association for Computational Linguistics.
- Van den Bosch, A. and T. Bogers. 2008. Efficient context-sensitive word completion for mobile devices. In *MobileHCI 2008: Proceedings of the 10th International Conference on Human-Computer Interaction with Mobile Devices and Services, IOP-MMI special track*, pages 465–470, Amsterdam, The Netherlands, September.
- Van den Bosch, A. and T. Bogers. 2013. Memory-based named entity recognition in tweets. In A. Cano, M. Rowe, M. Stankovic, and A.-S. Dadzie, editors, *#MSM2013 Workshop Concept Extraction Challenge Proceedings*, pages 40–43, Rio de Janeiro.
- Van den Bosch, A. and W. Daelemans. 1993. Data-oriented methods for grapheme-to-phoneme conversion. In *Proceedings of the 6th Conference of the EACL*, pages 45–53.
- Van den Bosch, A. and W. Daelemans. 1999. Memory-based morphological analysis. In *Proceedings of the 37th Annual Meeting of the ACL*, pages 285–292, San Francisco, CA. Morgan Kaufmann.
- Van den Bosch, A. and W. Daelemans. 2000. A distributed, yet symbolic model of text-to-speech processing. In P. Broeder and J.M.J. Murre, editors, *Models of Language Acquisition: inductive and deductive approaches*. Oxford University Press, pages 76–99.
- Van den Bosch, A. and W. Daelemans. 2013. Implicit schemata and categories in memory-based language processing. *Language and Speech*, 56(3):308–326.

- Van den Bosch, A., W. Daelemans, and A. Weijters. 1996. Morphological analysis as classification: an inductive-learning approach. In K. Oflazer and H. Somers, editors, *Proceedings of the Second International Conference on New Methods in Natural Language Processing, NeMLaP-2, Ankara, Turkey*, pages 79–89.
- Van den Bosch, A., E. Krahmer, and M. Swerts. 2001. Detecting problematic turns in human-machine interactions: Rule-induction versus memory-based learning approaches. In *Proceedings of the 39th Meeting of the Association for Computational Linguistics*, pages 499–506, New Brunswick, NJ. ACL.
- Van den Bosch, A., A. Risco Patón, T. Buijse, P. Berck, and M. Van Gompel. 2025. Memory-based language models: An efficient, explainable, and eco-friendly approach to large language modeling. *arXiv preprint arXiv:2510.22317*.
- Van den Bosch, A., N. Stroppa, and A. Way. 2007. A memory-based classification approach to marker-based EBMT. In F. Van Eynde, V. Vandeghinste, and I. Schuurman, editors, *Proceedings of the METIS-II Workshop on New Approaches to Machine Translation*, pages 63–72, Leuven, Belgium.
- Van den Bosch, A. and K. Van der Sloot. 2007. Superlinear parallelisation of the k -nearest neighbor classifier. In P. Adriaans, M. van Someren, and S. Katrenko, editors, *Proceedings of the 18th BENELEARN Conference*, Amsterdam, The Netherlands.
- Van den Bosch, A. and J. Zavrel. 2000. Unpacking multi-valued symbolic features and classes in memory-based language learning. In P. Langley, editor, *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 1055–1062, San Francisco, CA. Morgan Kaufmann.
- Van Gompel, M. 2010. Uvt-wsd1: A cross-lingual word sense disambiguation system. In *SemEval '10: Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 238–241, Morristown, NJ, USA. Association for Computational Linguistics.
- Van Gompel, M. and A. Van den Bosch. 2013. Wsd2: Parameter optimisation for memory-based cross-lingual word-sense disambiguation. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 183–187, Atlanta, Georgia, USA, June. Association for Computational Linguistics.
- Van Gompel, M. and A. Van den Bosch. 2014. Translation assistance by translation of L1 fragments in an L2 context. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 871–880, Baltimore, Maryland, June. Association for Computational Linguistics.
- Van Gompel, M., A. Van den Bosch, and P. Berck. 2009. Extending memory-based machine translation to phrases. In M. Forcada and A. Way, editors, *Proceedings of the Third Workshop on Example-Based Machine Translation*, pages 79–86, Dublin, Ireland.
- Van Halteren, H., J. Zavrel, and W. Daelemans. 2001. Improving accuracy in word class tagging through combination of machine learning systems. *Computational Linguistics*, 27(2):199–230.
- Van Rijsbergen, C.J. 1979. *Information Retrieval*. Butterworth, London.
- Van Zaanen, M. and Pieter Kanters. 2010. Automatic mood classification using tf*idf based on lyrics. In *11th International Society for Music Information Retrieval Conference (ISMIR 2010)*. pages 75–80.
- Vandekerckhove, B. 2013. *Exemplar-based generalisation at the interface between syntax and semantics*. Ph.D. thesis, University of Antwerp.

- Vandekerckhove, B., D. Sandra, and W. Daelemans. 2013. Selective impairment of adjective order constraints as overeager abstraction: An elaboration on kemmerer et al.(2009). *Journal of Neurolinguistics*, 26(1):46–72.
- Veenstra, J. 1998. Fast NP chunking using memory-based learning techniques. In *Proceedings of BENELEARN'98*, pages 71–78, Wageningen, The Netherlands.
- Veenstra, J., A. Van den Bosch, S. Buchholz, W. Daelemans, and J. Zavrel. 2000. Memory-based word sense disambiguation. *Computers and the Humanities*, 34(1/2):171–177.
- Verberne, S., A. Van den Bosch, H. Strik, and L. Boves. 2012. The effect of domain and text type on text prediction quality. In *Proceedings of the 13th Conference of the European Chapter of the Association for Computational Linguistics*, pages 561–569, New Brunswick, NJ. ACL.
- Weiss, S. and C. Kulikowski. 1991. *Computer systems that learn*. San Mateo, CA: Morgan Kaufmann.
- Wettschereck, D. 1994. *A study of distance-based machine learning algorithms*. Ph.D. thesis, Oregon State University.
- White, A.P. and W.Z. Liu. 1994. Bias in information-based measures in decision tree induction. *Machine Learning*, 15(3):321–329.
- Wilson, D.R. 1972. Asymptotic properties of nearest neighbor rules using edited data. *Institute of Electrical and Electronic Engineers Transactions on Systems, Man and Cybernetics*, 2:408–421.
- Witten, I. and E. Frank. 1999. *Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations*. San Mateo, CA: Morgan Kaufmann.
- Wolters, M. and A. Van den Bosch. 1997. Automatic phonetic transcription of words based on sparse data. In W. Daelemans, A. Van den Bosch, and A. Weijters, editors, *Workshop notes of the ECML/MLnet Familiarization Workshop on Empirical Learning of Natural Language Processing Tasks*, pages 61–70, Prague, Czech Republic. University of Economics.
- Wubben, S. 2010. Uvt: Memory-based pairwise ranking of paraphrasing verbs. In *Proceedings of the 5th International Workshop on Semantic Evaluation*, pages 260–263, Uppsala, Sweden.
- Wunsch, H., S. Kübler, and R. Cantrell. 2009. Instance sampling methods for pronoun resolution. In *Proceedings of the International Conference RANLP-2009*, pages 478–483, Borovets, Bulgaria, September. Association for Computational Linguistics.
- Yeh, A. 2000. Comparing two trainable grammatical relations finders. In *Proceedings of the 18th International Conference on Computational Linguistics*, pages 1146–1150, Saarbrücken, Germany.
- Zavrel, J. 1997. An empirical re-examination of weighted voting for k -NN. In W. Daelemans, P. Flach, and A. Van den Bosch, editors, *Proceedings of the 7th Belgian-Dutch Conference on Machine Learning*, pages 139–148, Tilburg.
- Zavrel, J., P. Berck, and W. Lavrijsen. 2000. Information extraction by text classification: Corpus mining for features. In *Proceedings of the workshop Information Extraction meets Corpus Linguistics*, Athens, Greece.
- Zavrel, J. and W. Daelemans. 1997. Memory-based learning: Using similarity for smoothing. In *Proceedings of the 35th Annual Meeting of the Association for Computational Linguistics*, pages 436–443.
- Zavrel, J. and W. Daelemans. 1999. Recent advances in memory-based part-of-speech tagging. In *VI Simposio Internacional de Comunicacion Social*, pages 590–597.

- Zavrel, J. and W. Daelemans. 2003. Feature-rich memory-based classification for shallow NLP and information extraction. In G. Nakhaeizadeh J. Franke and I. Renz, editors, *Text Mining, Theoretical Aspects and Applications*, pages 33–54, Heidelberg, Germany. Springer Physica-Verlag.
- Zavrel, J., W. Daelemans, and J. Veenstra. 1997. Resolving PP attachment ambiguities with memory-based learning. In M. Ellison, editor, *Proceedings of the Workshop on Computational Language Learning (CoNLL'97)*, ACL, Madrid.
- Zee, T., L. ten Bosch, I. Plag, and M. Ernestus. 2021. Paradigmatic relations interact during the production of complex words: Evidence from variable plurals in Dutch. *Frontiers in Psychology*, 12:720017.
- Zhang, J. 1992. Selecting typical instances in instance-based learning. In *Proceedings of the International Machine Learning Conference 1992*, pages 470–479.
- Zhekova, D. and S. Kübler. 2011. Ubiu: A robust system for resolving unrestricted coreference. In *Proceedings of the Fifteenth Conference on Computational Natural Language Learning: Shared Task*, pages 112–116, Portland, Oregon, USA, June. Association for Computational Linguistics.